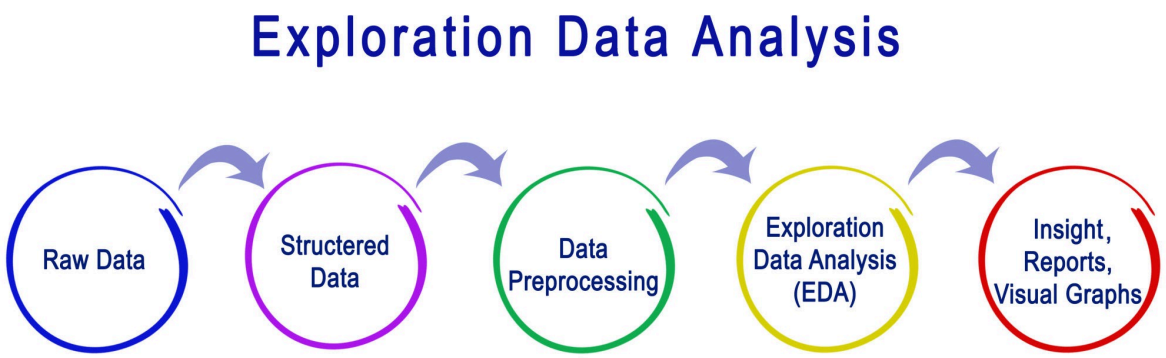
1. Introducción al Análisis Exploratorio de Datos..... 2
2. Visualización de datos con Matplotlib..... 3
2.1. Fundamentos de Matplotlib..... 4
2.2. Tipos de gráficas..... 7
2.2.1. Gráfico de Dispersión (nube de puntos)..... 7
2.2.2. Gráfico de Líneas..... 8
2.2.3. Gráfico de Barras..... 9
2.2.4. Histograma..... 12
2.2.5. Gráfico de Tarta..... 12
2.2.6. Gráfico de Hexágonos..... 13
2.2.7. Gráfico de Cajas (boxplots)..... 14
2.2.8. Otros gráficos..... 16
2.3. Configuraciones comunes..... 17
2.3.1. Identificación y Escalas (ax)..... 17
2.3.2. Elementos de Apoyo Visual (ax)..... 17
3.3.3. Gestión del Lienzo y Exportación (fig y plt)..... 18
2.4. Introducción a Librerías Modernas..... 18
2.4.1. Seaborn..... 18
2.4.1. Plotly..... 22
¿Qué aporta sobre Matplotlib?..... 22
Usos Principales..... 23
3. Estadística Descriptiva y su Representación..... 23
3.1. Medidas de Tendencia Central..... 24
3.2. Medidas de Dispersión..... 25
3.3. Posición y Cuantiles..... 28
3.4. Detección de Valores Atípicos (Outliers)..... 29
4. Análisis de Distribuciones y Relaciones..... 30
5. ANOVA..... 33
6. Análisis de Series Temporales..... 38
6.1. Preprocesado con Pandas..... 38
6.2. Transformación y suavizado..... 39
6.3. Autocorrelación..... 44

1. Introducción al Análisis Exploratorio de Datos

El Análisis Exploratorio de Datos (EDA) es la fase inicial crucial en ciencia de datos donde se examinan conjuntos de datos con gráficos y estadísticas para entender su estructura, identificar patrones, detectar anomalías (como datos faltantes o atípicos) y generar preguntas, en lugar de probar hipótesis, preparando los datos para análisis más profundos.



El EDA no es un paso mecánico, sino un proceso iterativo de investigación donde utilizamos herramientas estadísticas y visuales para:

1. **Comprender la estructura**: ¿Cuántas variables tenemos? ¿Son numéricas o categóricas?
2. **Visualización de datos:** Uso de gráficos (histogramas, diagramas de dispersión) para ver tendencias y distribuciones.
3. **Estadísticas descriptivas:** Calcular medidas como media, mediana, desviación estándar para resumir los datos
4. **Detección de problemas:** Identificar datos faltantes, duplicados, inconsistencias y valores atípicos (outliers).
5. **Generación de hipótesis:** Formular preguntas sobre relaciones entre variables, no confirmar hipótesis preexistentes.
6. **Preparación de datos:** Decidir cómo limpiar y transformar los datos para el análisis futuro
7. **Maximizar el conocimiento**: Descubrir relaciones subyacentes entre variables (correlaciones).

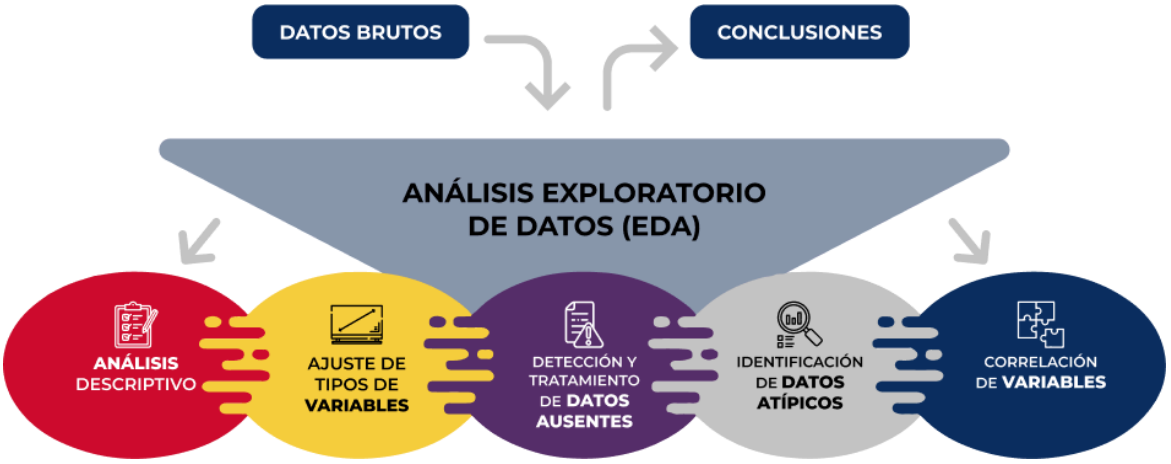


Figura 1: Etapas del EDA: Atribución

Para realizar el EDA utilizaremos:

- Las propias herramientas de consulta de valores nulos y repetidos, tipos de variables, etc que nos proporciona Pandas y se ha visto en temas anteriores.
- Visualización de datos (graficar datos) a través de librerías como Matplotlib, Seaborn o Plotly.
- Cálculos de medidas estadísticas y análisis de la varianza (librería statsmodels).

2. Visualización de datos con Matplotlib

Dentro del Análisis Exploratorio de Datos (EDA), la visualización no es un paso opcional, sino una herramienta fundamental para la **toma de decisiones estratégicas**. Un gráfico bien diseñado permite identificar patrones, tendencias y anomalías que quedarían ocultos en una tabla de datos pura.

Sin embargo, existe una diferencia crítica entre representar datos y comunicar información: mientras que generar una gráfica es una tarea técnica sencilla, diseñar una **visualización efectiva** —aquella que transmite un mensaje claro, preciso y libre de ambigüedades— requiere un equilibrio entre rigor estadístico y diseño funcional. En última instancia, estas gráficas no solo sirven para nuestro propio entendimiento, sino que se convierten en el vehículo principal para **comunicar hallazgos y facilitar la interpretación** de resultados a audiencias tanto técnicas como de negocio.

Vamos a analizar los tipos de gráficos más usuales. Aunque las librerías nos permiten representar gráficos en 3 dimensiones, nos centraremos en los gráficos de 2 dimensiones pues son los más útiles.

2.1. Fundamentos de Matplotlib

Matplotlib es la "librería abuela" de la visualización en Python. Es extremadamente potente y flexible, aunque su sintaxis puede parecer compleja al principio. Se integra bien con ndarrays de NumPy y Series de Pandas, ya que interpreta datos que estén almacenados en cualquier iterable lineal.

Nota: Matplotlib podría trabajar perfectamente dentro de un script, guardando las gráficas en un fichero imagen como png o jpg. Pero como es habitual en Análisis de Datos trabajaremos con documentos Jupyter, ya sea en Jupyter Lab o en VS Code, ya que de esta manera podemos visualizar las gráficas a medida que vamos analizando los datos a medida que escribimos el código.

Matplotlib:

- Requiere instalación: `pip install matplotlib`
- Enlace a PYPI: https://pypi.org/project/matplotlib/
- Web oficial: https://matplotlib.org/
- Documentación oficial: https://matplotlib.org/stable/index.html
- Cheatsheets oficiales: https://matplotlib.org/cheatsheets/

Matplotlib ofrece dos formas de trabajar (una **orientada a funciones**, más simple, y otra **orientada a objetos,** con mayor control). Para este documento, nos centraremos en el enfoque orientado a objetos, que es el más robusto para crear gráficos complejos y personalizados.

- **Orientado a funciones:** Todos los tipos de gráficos están disponibles a través de funciones en la librería **pyplot**. Por ejemplo, plt.plot() para gráfico de línea. No almacenamos ningún objeto. Todas las configuraciones de la gráfica se harán directamente a través de funciones de la librería pyplot.

```python
# Importación de la librería
import matplotlib.pyplot as plt
# Datos
x = [1, 2, 3, 4]
y = [1, 2, 3, 4]
# 2. Creación del gráfico (Enfoque Orientado a funciones)
plt.plot(x, y) # Creamos un gráfico de línea
plt.show() # Mostramos el gráfico
```

- **Orientado a objetos:** Almacenamos 2 tipos de objetos en 2 variables con los que interactuar para crear las gráficas.
    - El Objeto `fig` (Figure): Es el contenedor global. Controla aspectos como el tamaño total, el color de fondo de la ventana y la capacidad de guardar el archivo. Y sobre todo, el número de gráficas que contiene (una o varias).
    - El Objeto `ax` (Axes): Representa cada gráfica dentro de un fig. Una figura (`fig`) puede contener múltiples `ax` (subgráficos).

```python
# 1. Figure de una sola gráfica (Definimos tamaño global)
fig, ax = plt.subplots(figsize=(8, 4))
# 2. ax es el objeto sobre el que crear un gráfico
ax.plot(x, y) # Gráfico de línea
plt.show()
```

Al crear la figura (fig), podemos definir su fisonomía mediante atributos clave:

- `figsize=(ancho, alto)`: Define las dimensiones de la figura en pulgadas. Es vital para ajustar la proporción (por ejemplo, `(10, 5)` para una gráfica panorámica).
- `dpi` (Dots Per Inch): Controla la resolución. Un DPI de 100 o 300 es estándar para documentos e impresiones de alta calidad.
- `facecolor`: Permite cambiar el color de fondo del lienzo (por defecto es blanco).

Como hemos comentado, un figure puede tener varias gráficas. Cuando el análisis requiere comparar varias variables (por ejemplo, comparar ventas, beneficios y número de

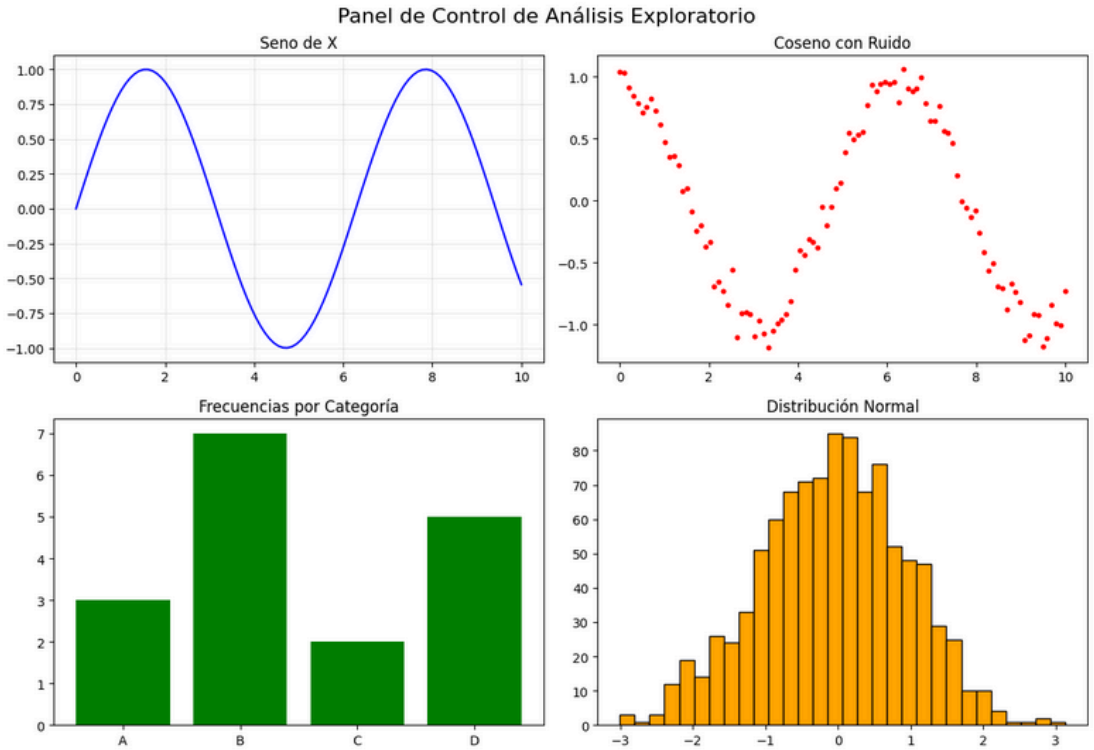
clientes), no es eficiente crear figuras separadas. La creación de una **cuadrícula de subplots** (subgráficas) permite organizar la información de forma compacta.

La sintaxis es `fig, ax = plt.subplots(n_filas, n_columnas)`

Dependiendo de la cantidad de gráficas, ax puede ser un solo objeto (si es 1x1), un **vector** si es una sola fila/columna, o una **matriz** si hay varias filas y columnas. Para dibujar en una gráfica específica, debemos indexar el objeto ax como si fuera una lista o una matriz de Python:

- **Vector (1 fila):** `ax[0]`, `ax[1]`, etc.
- **Matriz (Filas y Columnas):** `ax[0, 0]` (primera fila, primera columna), `ax[0, 1]` (primera fila, segunda columna).

```python
# Creamos la cuadrícula (2 filas y 2 columnas)
fig, ax = plt.subplots(2, 2, figsize=(12, 8), dpi=100)
ax[0, 0].plot()    # Gráfico [0, 0] - Superior Izquierda
ax[0, 1].scatter() # Gráfico [0, 1] - Superior Derecha
ax[1, 0].bar()     # Gráfico [1, 0] - Inferior Izquierda
ax[1, 1].hist()    # Gráfico [1, 1] - Inferior Derecha
```



2.2. Tipos de gráficas

Vamos a analizar los diferentes tipos de gráficas mostrando ejemplos realizados a través del método orientado a objetos de matplotlib.

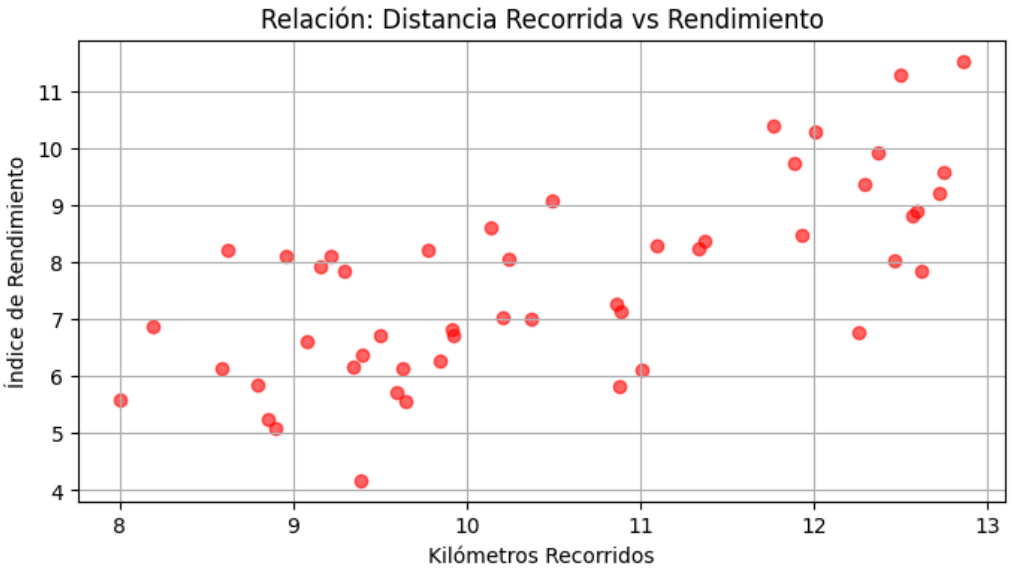
Los distintos tipos de gráficos vistos a continuación están igualmente disponibles a través del método orientado a funciones de la librería pyplot.

2.2.1. Gráfico de Dispersión (nube de puntos)

Muestra la relación (o correlación) entre **dos variables numéricas,** una representada en el eje X y otra representada en el eje Y. Cada observación en el conjunto de datos (x, y) representa un punto en el plano bidimensional.

- **Cuándo usarlo:** Para identificar si existe una relación entre variables (ej: ¿a más horas de estudio, mejores notas?). Permite detectar visualmente, tendencias lineales, agrupamientos (clusters) o correlaciones.
- **Ejemplo:** Relación entre los kilómetros recorridos por un futbolista y su rendimiento físico al final del partido.

```python
ax.scatter(x, y, color='red', alpha=0.6)
```



Algunos parámetros del método scatter:

- **color**: Color del punto.
- **alpha**: transparencia del punto.

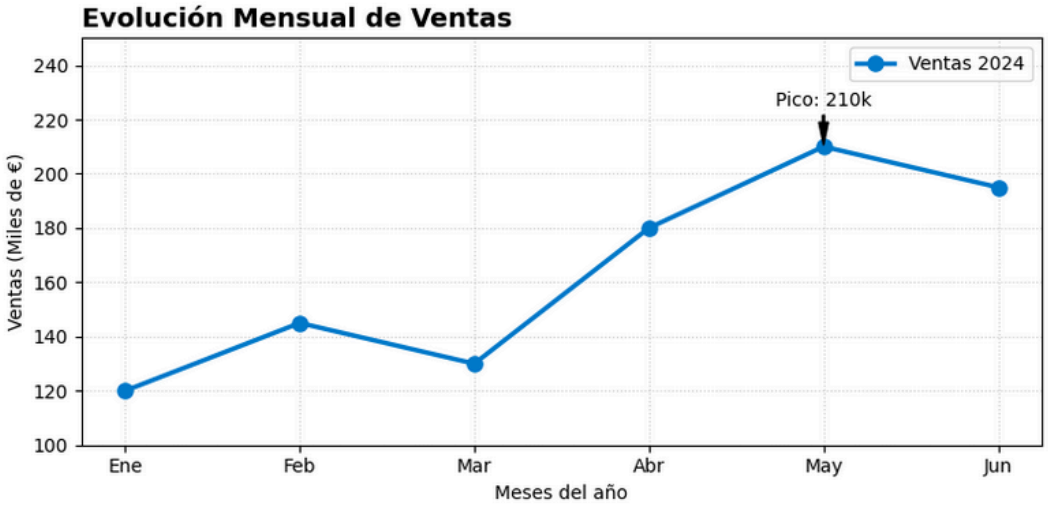
- **label**: Permite añadir una leyenda si se incluyen datos en diferentes colores invocando varias veces scatter sobre el mismo ax.
- **marker**: Tipo de marca/punto.

2.2.2. Gráfico de Líneas

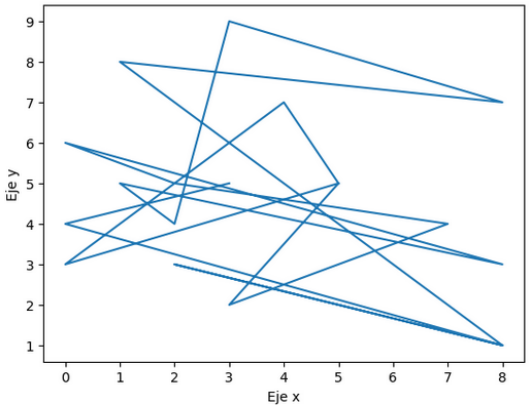
Representa la evolución de una variable numérica frente a otra (el tiempo si es una serie temporal). Su objetivo es mostrar **tendencias, ciclos o cambios** a lo largo de un intervalo continuo de tiempo o la relación de cambio entre 2 variables (como cambia una variable a medida que cambia la otra). Igual que en la nube de puntos, se representan 2 variables (x, y) en forma de puntos, pero posteriormente cada punto es unido con una línea en orden de aparición del eje X.

- **Cuándo usarlo:** Cuando el orden de los datos en el eje X tiene un significado (ej: cronología). Es ideal para ver si algo sube, baja o se mantiene estable.
- **Ejemplo:** Tendencia mensual de ventas en miles de euros (eje Y). El eje X es el tiempo en meses.

```python
ax.plot(x, y, marker='o', linestyle='-', color='blue',label='Ventas 2024')
```



El gráfico de línea requiere que el eje X esté ordenado de menor a mayor para que la línea no de saltos de izquierda a derecha.



Algunos parámetros del método plot:

- **color**: color de línea
- **linewidth**: ancho linea
- **alpha**: nivel de transparencia
- **linestyle**: estilo de línea
- **marker**: estilo del punto del dato
- **markersize**: tamaño del punto
- **markeredgewidth**: grosor línea del dato

El elemento “pico 210k” del gráfico es una anotación realizada con el **método “annotate”**

```python
ax.annotate(
  f'Pico 210k', # texto a mostrar
  xy=(ejex, ejey), # Punto del gráfico a donde apunta la flecha
  xytext=(posx, posy), # Posición del texto mostrado
  # Color y ancho del símbolo apuntador (flecha)
  arrowprops=dict(facecolor='black', width=1, headwidth=5))
```

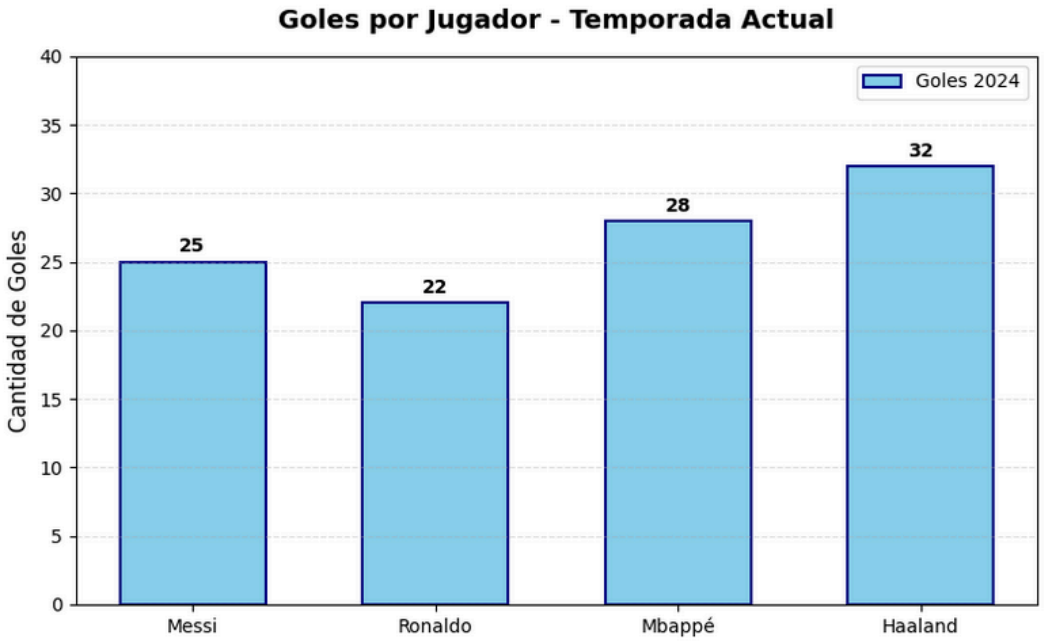
2.2.3. Gráfico de Barras

Compara una variable numérica (eje Y) a través de diferentes **categorías discretas** (eje X). La altura de la barra es proporcional al valor que representa.

- **Cuándo usarlo:** Para comparar magnitudes entre grupos que no tienen una relación de continuidad (ej: países, departamentos de una empresa, tipos de productos).

- **Ejemplo:** Comparativa de la cantidad de goles anotados por diferentes jugadores en una temporada.

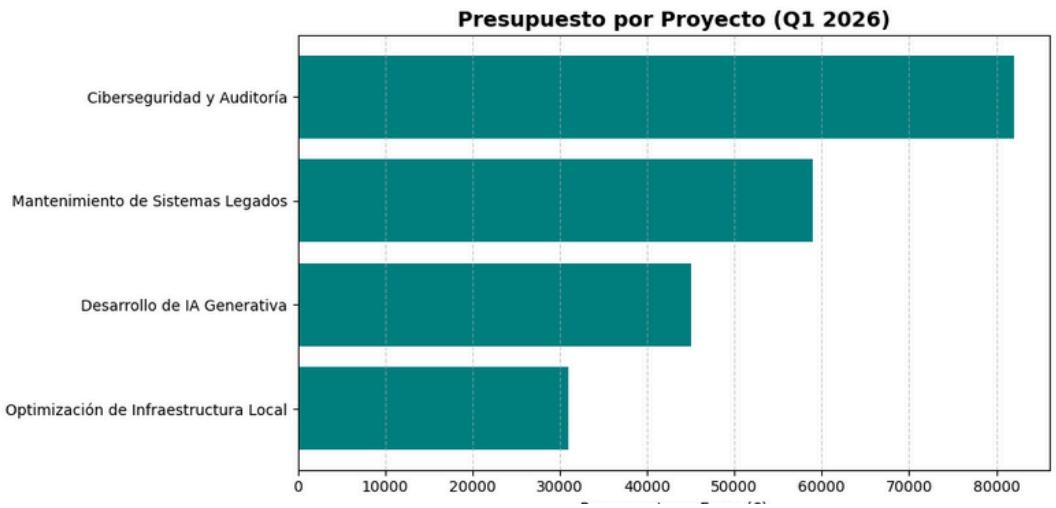
```python
ax.bar(jugadores, goles,
                color='skyblue',    # color del relleno
                edgecolor='navy',   # color del borde
                width=0.6,          # Baras un poco más delgadas
                linewidth=1.5,      # Grosor del borde
                label='Goles 2024') # etiqueta para leyenda
# Añadir etiquetas de valor sobre cada barra
# Iteramos sobre las barras creadas para poner el número exacto encima
ax.bar_label(barras, padding=3, fontsize=10, fontweight='bold')
```



El gráfico representado es un diagrama de barras verticales. Otras variantes son:

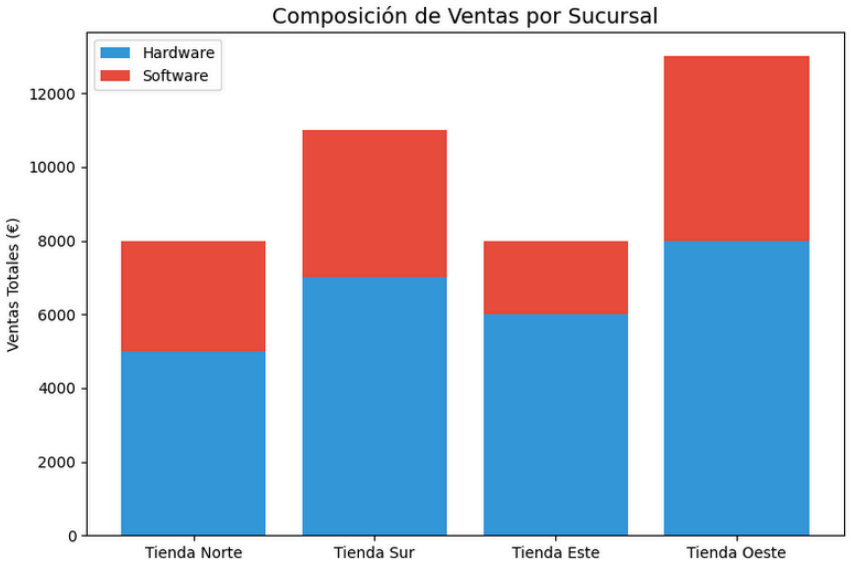
- Barras **horizontales**: Ideal para mostrar rankings, ya que el ojo humano lee de arriba hacia abajo.

```python
# Usamos barh en lugar de bar
ax.barh(proyectos, presupuesto, color='teal')
```



- Barras **apiladas:** Permite dividir cada barra en sub-segmentos, donde cada segmento representa una sub-categoría. La altura total de la barra muestra el total acumulado, mientras que los colores internos muestran la composición.

```python
# Dibujamos la primera capa
ax.bar(tiendas, hardware, label='Hardware', color='#3498db')
# Dibujamos la segunda capa encima (usando el parámetro bottom)
ax.bar(tiendas,software,bottom=hardware, label='Software', color='#e74c3c')
```

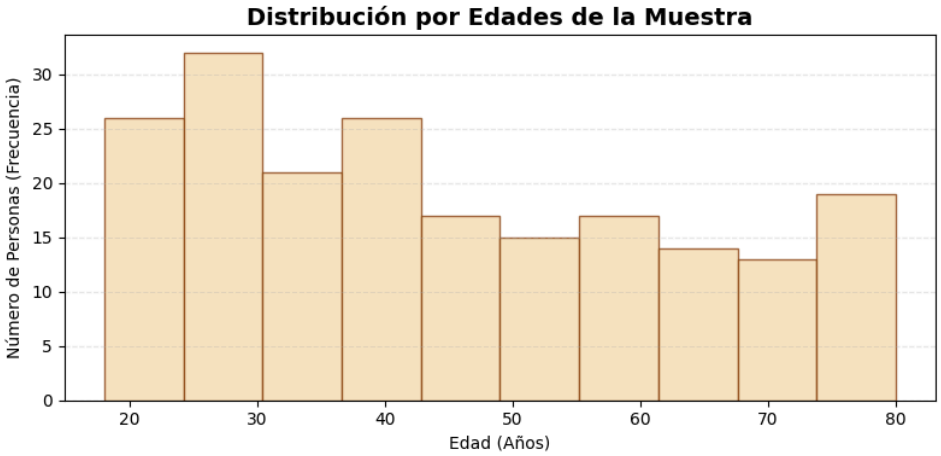


2.2.4. Histograma

A diferencia del gráfico de barras, el histograma representa la **distribución de una sola variable numérica**. Divide los datos en intervalos (llamados *bins*) y cuenta cuántas observaciones caen en cada uno. El eje X son los grupos realizados, la altura de cada barra representa cuántos individuos hay en ese grupo (frecuencia).

- **Cuándo usarlo:** Para entender la "forma" de tus datos: si están concentrados en el centro (distribución normal), si tienen sesgo hacia un lado o si hay múltiples picos.
- **Ejemplo:** Distribución de las edades de los clientes de un banco.

```python
# Agrupamos las edades en bloques
ax.hist(edades, bins=10, color='wheat', edgecolor='saddlebrown', alpha=0.8)
```



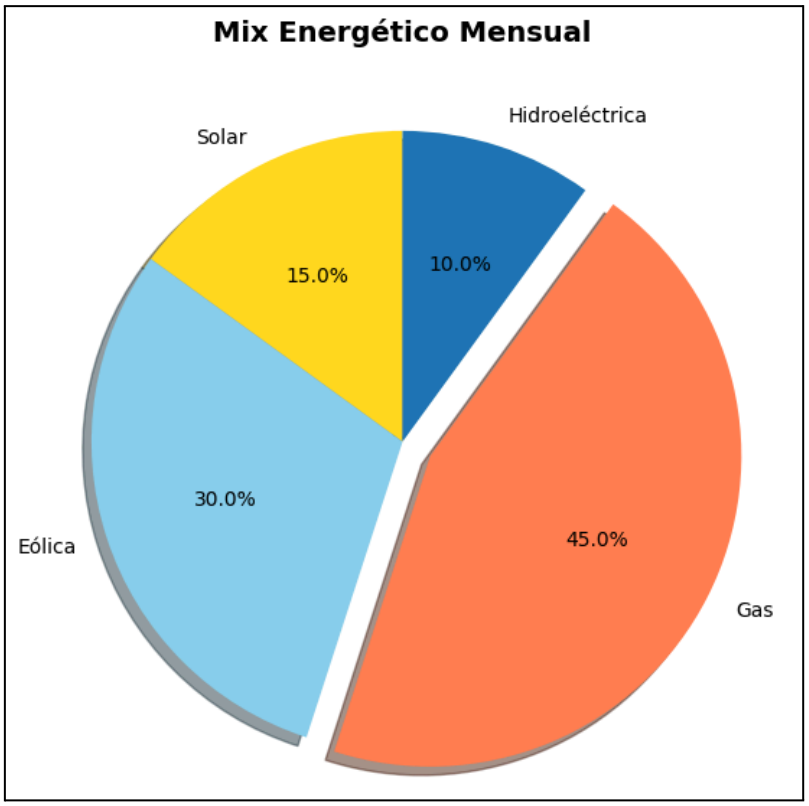
2.2.5. Gráfico de Tarta

Divide un círculo en sectores cuyas áreas son proporcionales a las cantidades que representan.

- **Cuándo usarlo:** Para mostrar **partes de un todo**. Es ideal cuando el número de categorías es muy pequeño (máximo 5) y queremos resaltar una proporción dominante.
- **Ejemplo:** Distribución de las fuentes de energía en una ciudad.

```python
# autopct: muestra el porcentaje con un decimal
```

```
# startangle: gira el gráfico para mejor visualización
# explode: separa un trozo para destacarlo
# shadow: Añade una sombra para dar efecto 3d
ax.pie(lista_cantidades, labels=fuentes, autopct='%1.1f%%', startangle=90,
       colors=colores, explode=(0, 0, 0.1, 0), shadow=True)
```

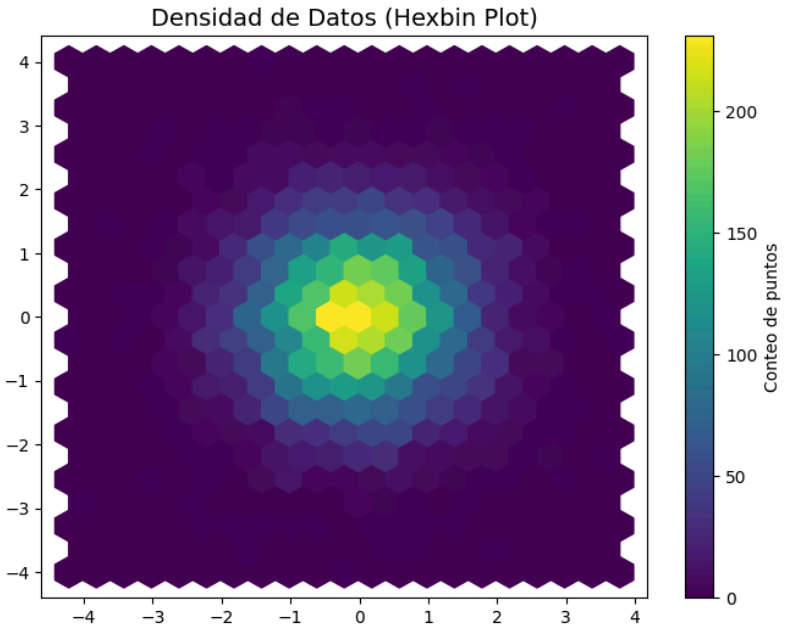


2.2.6. Gráfico de Hexágonos

Es una alternativa al gráfico de dispersión (scatter o nube de puntos) para **grandes volúmenes de datos**. Divide el plano en hexágonos y el color de cada uno indica la densidad de puntos en esa zona.

- **Cuándo usarlo:** Cuando tienes miles de puntos y un scatter plot normal se vería como una mancha sólida de color donde no se distingue nada (**overplotting**).
- **Ejemplo:** Densidad de población por coordenadas o relación entre miles de transacciones bancarias.

```python
grafica = ax.hexbin(x, y, gridsize=20) # gridsize: tamaño de los hexágonos
# Añade la escala de color
fig.colorbar(grafica, ax=ax, label='Conteo de puntos')
```

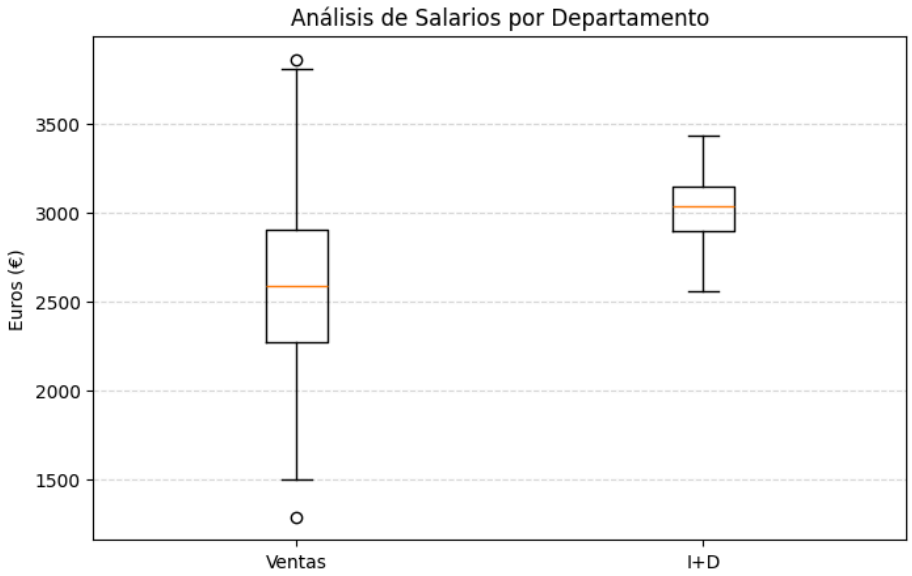


2.2.7. Gráfico de Cajas (boxplots)

Resume la distribución de los datos mediante sus **cuartiles**. Muestra la mediana, el rango intercuartílico (la caja) y los valores atípicos (los puntos fuera de los "bigotes").

- **Cuándo usarlo:** Es la herramienta reina para detectar **Outliers** y comparar la dispersión entre diferentes grupos de forma compacta. Los veremos en más profundidad en secciones posteriores.
- **Ejemplo:** Comparativa de salarios entre diferentes departamentos.

```python
# Los datos a mostrar van en una lista de arrays/series
# Cada array será una caja
# tick_labels son las etiquetas de cada caja
ax.boxplot([dept_ventas, dept_id], tick_labels=['Ventas', 'I+D'])
```



La Anatomía del Boxplot

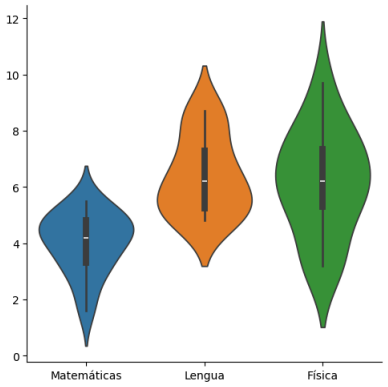
Dados los sueldos de un departamento, ordenados de menor a mayor, el boxplot divide esos datos en cuatro partes iguales (25% cada una), cuartiles.

- **La Línea Central (Mediana / Q2):** Es la línea que divide la caja. Representa el valor que queda justo en medio de los datos. El 50% de los datos es menor que este valor y el otro 50% es mayor.
- **La Caja (Rango Intercuartílico - IQR)** representa el 50% central de los datos:
    - **Límite Inferior (Q1):** Es el **Primer Cuartil**. El 25% de los datos están por debajo de este punto.
    - **Límite Superior (Q3):** Es el **Tercer Cuartil**. El 75% de los datos están por debajo de este punto.
    - **Cuerpo de la caja:** Contiene el **50% central** de tus datos. Cuanto más larga sea la caja, más dispersos están los datos "típicos".
- **Los Bigotes (Whiskers):** Son las líneas que se extienden fuera de la caja.
    - Representan los valores que están dentro de un rango "normal". Generalmente, llegan hasta el valor máximo y mínimo que **no** sean considerados atípicos.
    - Técnicamente, suelen calcularse como 1.5×IQR por encima de Q3 y por debajo de Q1.
- **Puntos Fuera de los Bigotes (Outliers):** Son los **valores atípicos**. Si ves puntos o círculos aislados más allá de los bigotes, significa que esos datos son inusualmente altos o bajos comparados con el resto.

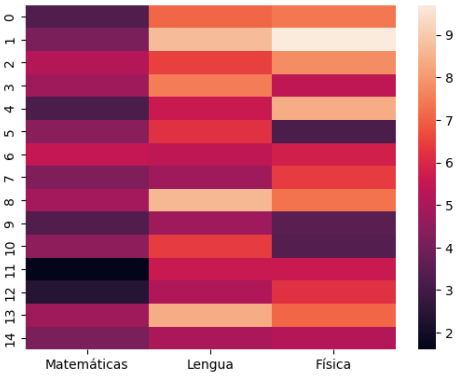
2.2.8. Otros gráficos

Existen más tipos de gráficos y otras variantes de los vistos de las que no entraremos a estudiarlos en profundidad, ya que requeriría mucho tiempo y además, puede ser más eficiente realizarlas con otras librerías. Algunos ejemplos son:

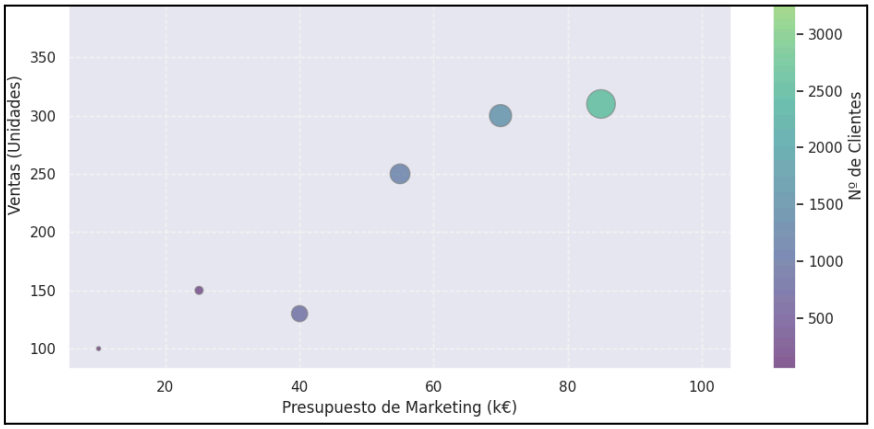
- Diagrama de violín (violinplot): Muestra la misma información que el boxplot pero añade la "forma" de la distribución (dónde hay más densidad de datos).



- Mapa de calor: Representa una matriz de valores mediante colores. Cada celda de la matriz corresponde a una intensidad de color según su valor numérico.



- Gráfico de burbujas: Es similar a una nube de puntos, pero ahora los puntos tienen tamaño en función del valor de una tercera variable.



Una lista más exhaustiva puede encontrarse en la [documentación de matplotlib.]

2.3. Configuraciones comunes

Aparte de la configuración particular de cada tipo de gráfico, hay múltiples elementos comunes. Estas configuraciones están disponibles tanto en la filosofía orientada a funciones como en la orientada a objetos.

En el enfoque orientado a objetos, la mayoría de las funciones de personalización se ejecutan a través del objeto `ax` (Axes) o del objeto `fig` (Figure). A continuación, se detallan los elementos comunes que garantizan una visualización clara y profesional.

2.3.1. Identificación y Escalas (ax)

- **`ax.set_title()`**: Define el título principal del gráfico.
    - *Atributos útiles:* `fontsize` (tamaño), `fontweight` (grosor, ej: 'bold'), y `pad` (espacio entre el título y el gráfico).
- **`ax.set_xlabel()`** / **`ax.set_ylabel()`**: Establecen las etiquetas de los ejes X e Y. Son fundamentales para que el lector sepa qué magnitudes está observando.
- **`ax.set_xlim()`** / **`ax.set_ylim()`**: Definen los límites mínimos y máximos de los ejes. Muy útil para eliminar espacios en blanco innecesarios o para normalizar escalas en comparativas.
- **`ax.set_xticks()`** / **`ax.set_yticks()`**: Permiten definir manualmente dónde aparecerán las marcas de graduación en los ejes.

2.3.2. Elementos de Apoyo Visual (ax)

- **`ax.legend()`**: Despliega la leyenda del gráfico. Solo funciona si previamente hemos definido el parámetro `label` dentro de las funciones de dibujo (como `plot` o `bar`). *Tip:* Usa `loc='best'` para que Matplotlib busque automáticamente el lugar donde menos estorbe.
- **`ax.grid()`**: Activa la rejilla de fondo.
    - *Atributos útiles:* `axis='y'` (solo líneas horizontales), `linestyle=':'` (punteada) y `alpha` (transparencia) para que no distraiga de los datos.
- **`ax.axhline()`** / **`ax.axvline()`**: Dibujan una línea infinita horizontal o vertical, respectivamente. Ideales para marcar la **media**, un valor objetivo o un límite crítico.

- **`ax.axhspan()`** / **`ax.axvspan()`**: Crean una franja o sombreado rectangular que cubre un rango del eje. Se usa para resaltar zonas de interés, como periodos de crisis en una serie temporal o rangos de valores normales en un análisis médico.
- **`ax.fill_between()`**. Sirve para pintar una zona sombreada similar al anterior pero de forma dinámica, en función de los datos. Esta función depende de las coordenadas x e y. El área se va ensanchando o estrechando según los valores de tus listas o arrays. Útil para sombrear la incertidumbre o el error alrededor de una línea de datos.

3.3.3. Gestión del Lienzo y Exportación (fig y plt)

- **`plt.tight_layout()`**: Es un comando de "limpieza". Ajusta automáticamente los subplots para que las etiquetas de los ejes y los títulos no se solapen entre sí ni con los bordes de la figura.
- **`plt.suptitle()`**: Añade un título global centrado en la parte superior de la figura, por encima de todos los subplots.
- **`fig.savefig()`**: El método para exportar nuestra figura a un archivo en disco. Usar `bbox_inches='tight'` para asegurar que no se corte nada. El formato del archivo se incluye en el nombre.

2.4. Introducción a Librerías Modernas

2.4.1. Seaborn

Seaborn está construido sobre Matplotlib y está diseñado específicamente para trabajar con **Pandas**. Su gran ventaja es que permite realizar visualizaciones complejas con poquísimas líneas de código, con interpretación y visualización automática de artefactos estadísticos (por ejemplo, rectas de regresión en una nube de puntos).

- Requiere instalación: `pip install seaborn`
- Enlace a PYPI: https://pypi.org/project/seaborn/
- Web oficial: https://seaborn.pydata.org/
- Documentación oficial: https://seaborn.pydata.org/api.html
- Galería de gráficas: https://seaborn.pydata.org/examples/index.html

¿Qué aporta?

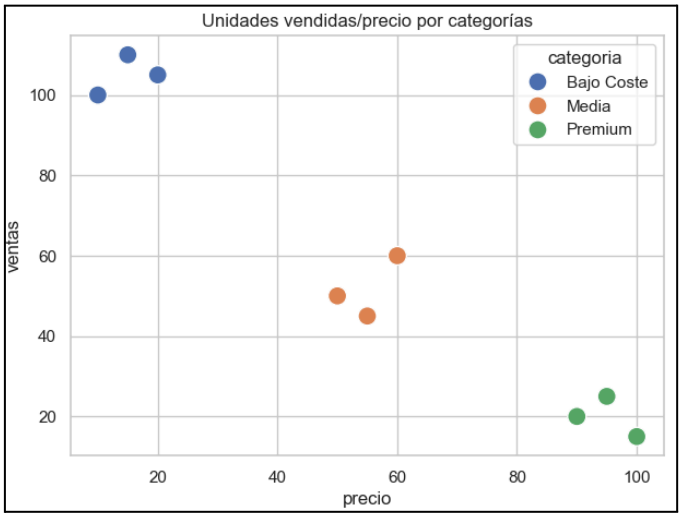
- **Estética superior:** Paletas de colores modernas y diseños más limpios por defecto, sin necesidad de configurar manualmente tantos parámetros.
- **Enfoque estadístico:** Muchas funciones calculan automáticamente intervalos de confianza, medias y regresiones lineales.
- **Integración con DataFrames:** Entiende directamente las columnas.

La filosofía de Seaborn

Mientras que en Matplotlib tú le dices: *"Dibuja una nube de puntos con estos datos en el eje x y estos otros en el eje y*, en **Seaborn** le dices: *"Toma este **Dataset** y analiza la relación entre la columna Edad y la columna Salario, separando por Género"*.

Diferencias Clave:

1. **Enfoque en Datos (Pandas)**: Seaborn está diseñado para entender DataFrames de Pandas directamente. No necesitas extraer las columnas en listas; solo pasas el nombre de la columna como un texto.



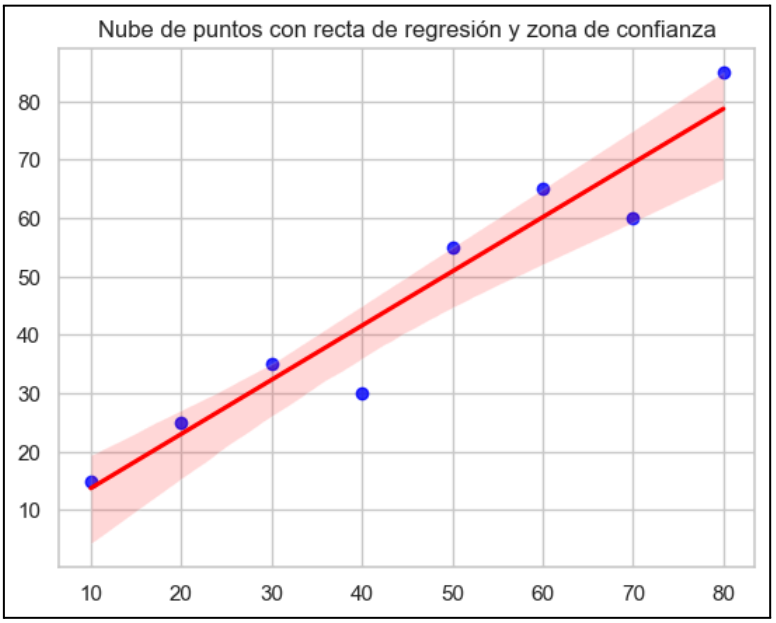
Con Matplotlib:

```
fig, ax = plt.subplots(figsize=(7, 5))
# Tenemos que dibujar cada grupo por separado para controlar el color y la etiqueta
for cat, color in zip(['Bajo Coste', 'Media', 'Premium'], ['blue', 'green', 'red']):
    filtro = df[df['categoria'] == cat]
    # Cada color/grupo es una invocación de scatter
    ax.scatter(filtro['precio'], filtro['ventas'], c=color, label=cat, s=100)
```

Con Seaborn:

```
fig, ax = plt.subplots(figsize=(7, 5))
# Parámetro hue indica columna del DF para dividir en grupos
sns.scatterplot(data=df, x='precio', y='ventas', hue='categoria', s=150, ax=ax)
```

2. **Menos Código, más Estadística**: Una sola función de Seaborn puede hacer lo que en Matplotlib requiere 10 líneas (como calcular intervalos de confianza o ajustar líneas de regresión).



Con Matplotlib:

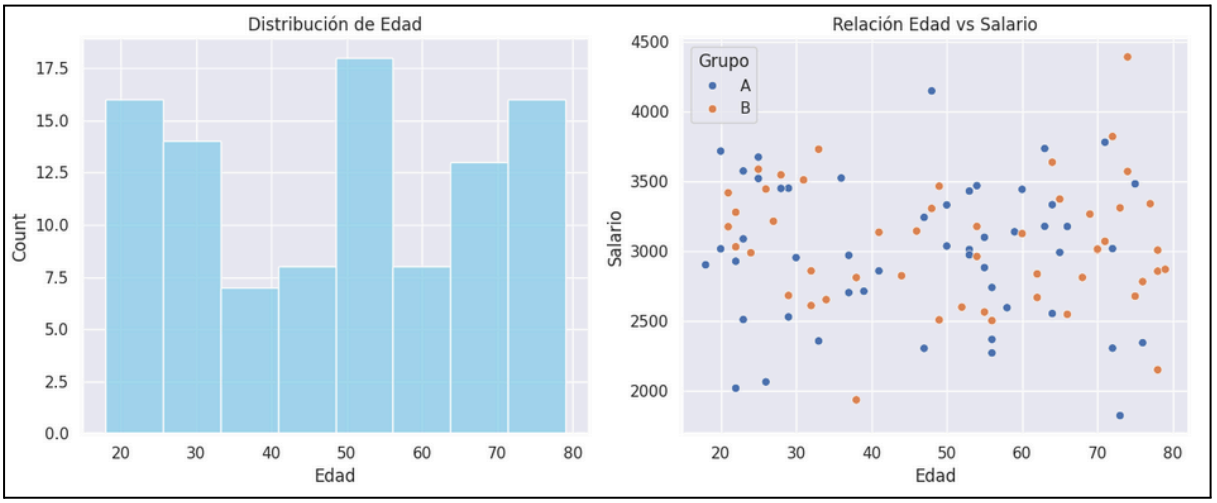
```
# 1. Calcular la línea de regresión (y = mx + b)
m, b = np.polyfit(x, y, 1)
# 2. Calcular el intervalo de confianza (esto es una versión simplificada, la real son muchas más líneas)
y_pred = m * x + b
error = np.std(y - y_pred) # Estimación simple del error
# 3. Dibujar puntos
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(x, y, color='blue', label='Datos')
# 4. Dibujar línea de regresión
ax.plot(x, y_pred, color='red', label='Ajuste lineal')
# 5. Dibujar intervalo de confianza (sombreado)
ax.fill_between(x, y_pred - error, y_pred + error, color='red', alpha=0.2, label='Confianza')
```

Con Seaborn:

```
# Todo lo anterior se resume en UNA sola función:
# ci=95: Calcula automáticamente el intervalo de confianza al 95%
sns.regplot(x=x, y=y, ci=95, scatter_kws={'color':'blue'}, line_kws={'color':'red'})
```

3. **Híbrido con Matplotlib**: Seaborn no reemplaza a Matplotlib; **se apoya en él**. De hecho, puedes crear un ax con Matplotlib y decirle a Seaborn que dibuje dentro de ese "cuadro".

```
# 1. Creamos datos de ejemplo en un DataFrame (Estilo Seaborn)
data = pd.DataFrame({
    'Edad': np.random.randint(18, 80, 100),
    'Salario': np.random.normal(3000, 500, 100),
    'Grupo': np.random.choice(['A', 'B'], 100)})
# 2. Definimos la estructura con Matplotlib (1 fila, 2 columnas)
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
# 3. Usamos Seaborn para "rellenar" cada ax
# Usamos el parámetro 'ax' para decirle dónde debe dibujar
sns.histplot(data=data, x='Edad', ax=ax[0], color='skyblue')
ax[0].set_title('Distribución de Edad')
sns.scatterplot(data=data, x='Edad', y='Salario', hue='Grupo', ax=ax[1])
ax[1].set_title('Relación Edad vs Salario')
# 4. Ajuste final con Matplotlib
plt.tight_layout()
plt.show()
```



Gráficas de Seaborn y breve descripción:

- `sns.scatterplot()`: Gráfico de dispersión. Ideal para ver la relación entre dos variables numéricas y agrupar por colores con el parámetro `hue`.
- `sns.lineplot()`: Gráfico de líneas. Perfecto para series temporales; gestiona automáticamente intervalos de confianza si hay múltiples observaciones por punto.
- `sns.histplot()`: Histograma. Representa la distribución de una variable. Permite añadir una curva de densidad suave con `kde=True`.
- `sns.barplot()`: Gráfico de barras. Por defecto calcula la media de los datos y añade una barra de error que indica la incertidumbre estadística.

- `sns.boxplot()`: Diagrama de caja y bigotes. Muestra los cuartiles, la mediana y los valores atípicos (outliers) de un vistazo.
- `sns.violinplot()`: Gráfico de violín. Combina un boxplot con una estimación de densidad, mostrando dónde se concentran más los datos.
- `sns.heatmap()`: Mapa de calor. Visualización matricial ideal para mostrar tablas de correlación mediante una escala de colores.
- `sns.regplot()`: Gráfico de regresión. Dibuja una nube de puntos y calcula automáticamente la línea de tendencia lineal y su intervalo de confianza.
- `sns.countplot()`: Gráfico de frecuencias. Cuenta automáticamente el número de apariciones de una variable categórica (no necesita cálculo previo).
- `sns.jointplot()`: Gráfico conjunto. Muestra la relación entre dos variables y sus distribuciones individuales en los márgenes de forma simultánea.
- `sns.pairplot()`: Gráfico de pares. Crea una matriz de gráficos de dispersión para todas las variables numéricas de un dataset de una sola vez.
- `sns.kdeplot()`: Gráfico de densidad de kernel. Visualiza la "forma" de los datos mediante una curva de probabilidad suavizada sin usar barras.

2.4.1. Plotly

Plotly es una librería de código abierto que permite crear gráficos interactivos que se ejecutan en el navegador. A diferencia de Matplotlib, que genera imágenes estáticas (como un `.png`), Plotly genera objetos basados en **JavaScript** con los que el usuario puede interactuar.

- Requiere instalación: `pip install plotly`
- Enlace a PYPI: https://pypi.org/project/plotly/
- Web oficial: https://plotly.com/python/
- Documentación oficial: https://plotly.com/python-api-reference/

¿Qué aporta sobre Matplotlib?

- **Interactividad Nativa:** Al pasar el ratón sobre un dato, aparece un cuadro de información (**tooltip**) con los valores exactos.
- **Exploración en Tiempo Real:** Permite hacer zoom, desplazar el eje y aislar grupos de datos haciendo clic en la leyenda sin necesidad de reprogramar nada.

- **Independencia de Resolución:** Los gráficos son vectoriales, lo que significa que no pierden calidad al redimensionarlos o ampliarlos.
- **Menos Código para Gráficos Complejos:** Tareas difíciles en Matplotlib (como un gráfico de burbujas animado o un mapa geográfico) se resuelven en Plotly con un solo comando gracias a su módulo **Plotly Express**.

Usos Principales

1. **Dashboards Profesionales:** Es la base de herramientas como *Dash*, utilizadas para crear paneles de control empresariales..
2. **Análisis de Outliers:** Al ser interactivo, es mucho más fácil identificar qué punto exacto es un valor atípico simplemente pasando el cursor sobre él.
3. **Visualización Geoespacial:** Excelente para mapas con coordenadas GPS o por países.
4. **Gráficos 3D:** Permite rotar y explorar superficies o nubes de puntos en tres dimensiones de forma fluida.
5. **Web:** Ideal para insertar gráficos en páginas web o reportes de Jupyter que el cliente pueda explorar por su cuenta.Se integra perfectamente con Django.

3. Estadística Descriptiva y su Representación

Una vez que sabemos visualizar datos, el siguiente paso en el EDA es resumirlos. No podemos mirar una tabla de 10.000 filas y entender qué pasa; necesitamos números que actúen como "embajadores" de todo el conjunto. El análisis de datos nos permite descubrir patrones, hacer predicciones y tomar decisiones informadas.

Por ejemplo: en el fútbol, los datos pueden responder preguntas como:

- ¿Los equipos que tienen más posesión del balón ganan más partidos?
- ¿Existe una relación entre la cantidad de tiros a puerta y la cantidad de goles anotados?
- ¿Los equipos que corren más kilómetros tienen mejor rendimiento?

3.1. Medidas de Tendencia Central

En estadística, la media, la mediana y la moda son medidas de **tendencia central** que nos ayudan a entender el centro de un conjunto de datos. Cada una de ellas ofrece una perspectiva diferente sobre dónde se encuentra el "**valor típico**" de los datos:

Media (Promedio):

- **Qué es**: Es el promedio aritmético de un conjunto de valores. Se calcula sumando todos los valores y dividiendo el resultado por el número total de valores.
- **Qué nos dice**: Nos da una idea del valor central general de los datos. Sin embargo, puede verse afectada por valores extremos (valores muy altos o muy bajos).

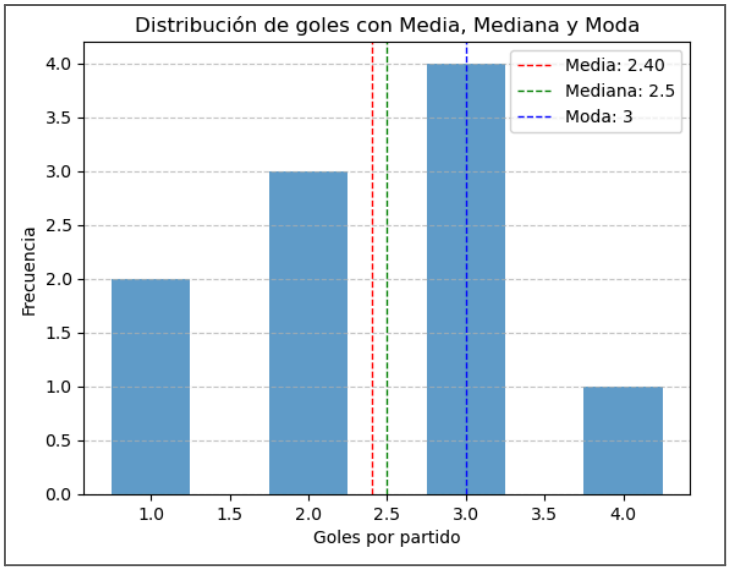
Mediana:

- **Qué es**: Es el valor que se encuentra en el centro de un conjunto de datos ordenados de menor a mayor. Si hay un número par de valores, la mediana es el promedio de los dos valores centrales.
- **Qué nos dice**: Es útil cuando queremos evitar la influencia de valores extremos. Nos da una idea del valor central que divide el conjunto de datos en dos partes iguales.

Moda:

- **Qué es**: Es el valor que aparece con mayor frecuencia en un conjunto de datos. Un conjunto de datos puede tener una moda (unimodal), dos modas (bimodal) o más modas (multimodal).
- **Qué nos dice**: La moda nos indica cuál es el valor más común en un conjunto de datos. Es especialmente útil para datos categóricos, ya que no es posible calcular la media.

Representación sobre **histogramas** para analizar el sesgo: En un histograma, puedes representar directamente la **Media** y la **Mediana** como líneas verticales/horizontales porque comparten la misma unidad que los datos (el eje X o Y según el caso).



Media vs. Mediana: El detector de Sesgo: La relación de posición entre estas dos medidas te dice hacia dónde "cojea" la distribución sin necesidad de calcular fórmulas complejas:

- **Distribución Simétrica:** Media ≈ Mediana. Los datos están equilibrados (como una Campana de Gauss perfecta).
- **Sesgo a la Derecha (Positivo): Media > Mediana**. Indica que hay valores extremadamente altos (outliers) que están "tirando" de la media hacia arriba, aunque la mayoría de los datos sean bajos. *Ejemplo: Los salarios en una empresa con 3 directivos que ganan muchísimo.*
- **Sesgo a la Izquierda (Negativo): Media < Mediana**. Hay valores extremadamente bajos que hunden el promedio. *Ejemplo: La edad de fallecimiento en un país desarrollado.*

3.2. Medidas de Dispersión

La varianza y la desviación estándar son medidas de **dispersión** que nos ayudan a entender **cuánto se alejan** los **datos** de la **media**.

Varianza

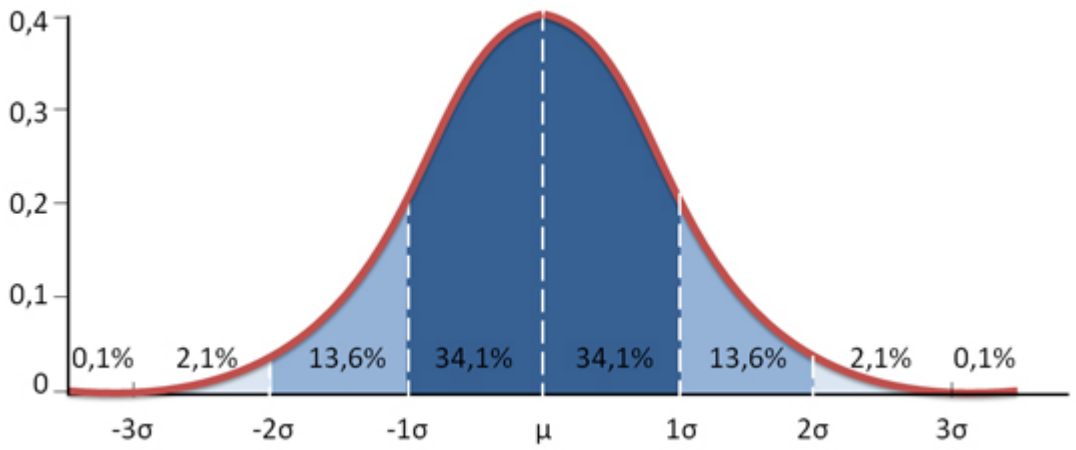
$$\sigma^2 = \frac{\Sigma (x_i - \mu)^2}{N}$$

- La varianza mide el promedio de los cuadrados de las diferencias entre cada valor y la media. En otras palabras, cuantifica cuánto se dispersan los datos alrededor de la media.
- Una **varianza alta** indica que los **datos están muy dispersos**, mientras que una **varianza baja** indica que están más **concentrados alrededor de la media.**

Desviación Estándar σ (sigma)

- La **desviación estándar es la raíz cuadrada de la varianza**. Nos da una medida de dispersión en las mismas unidades que los datos originales, lo que la hace **más fácil de interpretar que la varianza**.
- Al igual que la varianza, una desviación estándar alta indica una mayor dispersión de los datos.

Distribución normal o Campana de Gauss: Matemáticamente, la distribución normal es una función de probabilidad de una variable aleatoria continua. De forma más sencilla, es el modelo que describe cómo se reparten la mayoría de los fenómenos naturales y sociales (estaturas, errores de medición, puntajes en pruebas).



Regla Empírica (o regla 68.2 95.4 99.7) es la "guía rápida" que usamos en estadística para entender qué tan dispersos están los datos en una distribución normal sin tener que hacer cálculos complejos cada vez.

1. El primer nivel: 68.2% (1 sigma): Aproximadamente el 68% de los datos se encuentran a una distancia de 1 desviación estándar. En la práctica: Si el promedio de notas de una clase es 70 y la desviación es 10, el 68% de los alumnos sacó entre 60 y 80.

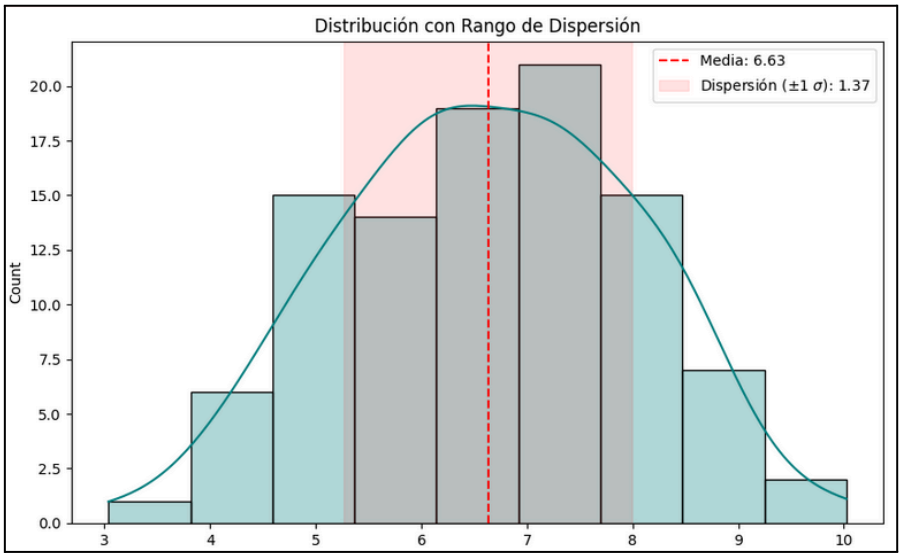
2. El segundo nivel: 95.4% (2 sigma): Aproximadamente el 95% de los datos se encuentran a 2 desviaciones estándar de la media. En la práctica: Siguiendo el ejemplo

anterior, el 95% de los alumnos sacó entre 50 y 90. Casi todos los casos "normales" o esperados entran aquí.

3. El tercer nivel: 99.7% (3 sigma): Aproximadamente el 99.7% de los datos se encuentran a 3 desviaciones estándar de la media. En la práctica: Es extremadamente raro encontrar un dato fuera de este rango (solo 3 de cada 1,000 casos). En el ejemplo, serían notas menores a 40 o mayores a 100.

En una gráfica es útil pintar la media y un rango alrededor de 1 sigma con el uso de `axhspan/axvspan` para visualizar la concentración de la mayoría de los datos y la dispersión respecto a la media.

- **Utilidad:** En distribuciones que se parecen a la "Normal" (campana de Gauss), el **68% de tus datos** deberían caer dentro del rango [media − σ, media + σ].
- **Diagnóstico visual:**
  - Si la franja es **estrecha**: Los datos son muy consistentes (poca variabilidad).
  - Si la franja es **ancha**: Los datos están muy dispersos; la media es "poco representativa" porque los valores reales están muy lejos de ella.



3.3. Posición y Cuantiles

Permiten saber dónde se sitúa un dato respecto al resto del grupo.

Percentil

Un percentil es un valor que divide un conjunto de datos ordenados en cien partes iguales. El percentil p indica que el p% de los datos son menores o iguales a ese valor.

Los percentiles se utilizan para entender la posición relativa de un valor dentro de un conjunto de datos. **Ejemplo**: Imagina que tenemos los goles marcados por un grupo jugadores en una temporada. Si un jugador está en el percentil 90, significa que:

a. El jugador supera al 90% de los demás jugadores: Esto quiere decir que el rendimiento del jugador es mejor que el de la gran mayoría.
b. Solo el 10% de los jugadores lo superan: Esto también implica que solo una pequeña fracción de los jugadores tiene un rendimiento superior en goles.

Podemos preguntarnos: ¿cuántos goles corresponden un percentil concreto? y lo contrario, ¿dado una cantidad de goles, cuál es su percentil?. También podemos preguntarnos ¿cuántos y cuáles son los futbolistas que están por encima o por debajo de tal percentil?

Cuartil

Un cuantil es similar a un percentil, pero en lugar de dividir los datos en cien partes, los divide en 4 partes. **(Q1, Q2, Q3):** Dividen los datos en cuatro cuartos (25% cada uno).

- **Q1 (Percentil 25):** El 25% de los datos está por debajo.
- **Q2 (Percentil 50):** Es la **Mediana**.
- **Q3 (Percentil 75):** El 75% de los datos está por debajo.

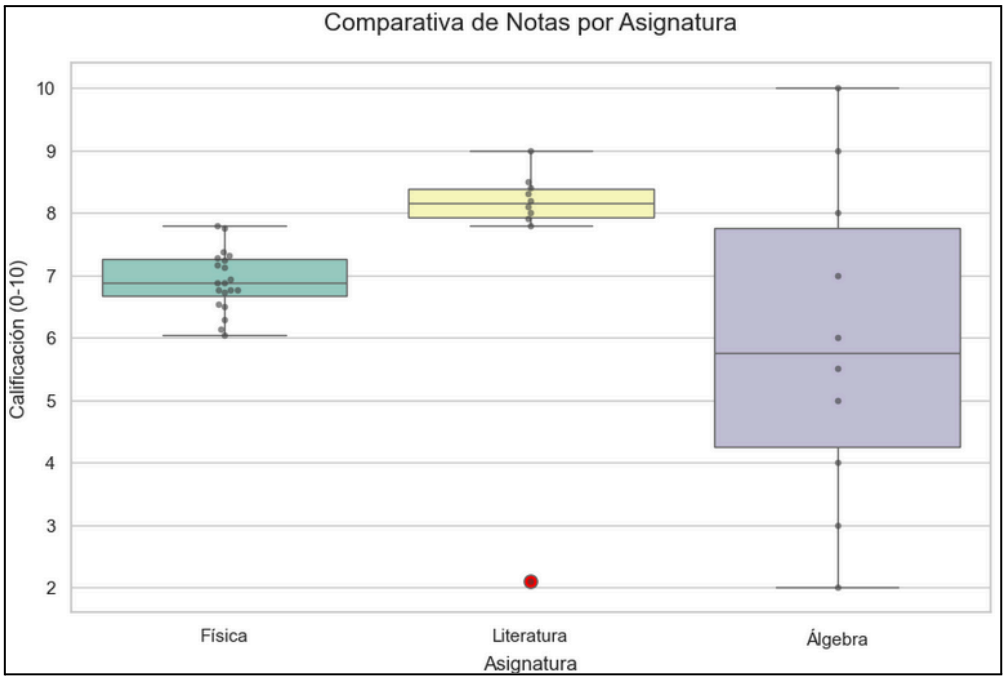
Ejemplo: Podemos usar cuartiles para dividir a los jugadores en cuatro grupos según sus goles marcados: el primer cuartil (25%) serían los que menos goles han marcado, y el cuarto cuartil (75-100%) los que más.

En **Pandas** y **Numpy** tanto los **percentiles** como **cuartiles** se calculan igual, mediante el método general `quantile(valor)`. El valor es un valor entre 0 y 1, siendo 0.5 el percentil 50%.

Cuartiles Rango Intercuartílico (IQR): Es la distancia entre Q3 y Q1 (IQR=Q3−Q1). Representa dónde se concentra el **50% central** de tus datos. Es la base para dibujar los Boxplots y detectar outliers.

3.4. Detección de Valores Atípicos (Outliers)

Un **Outlier** es una observación que se aleja tanto del resto que despierta sospechas de que fue generada por un mecanismo diferente (un error de medición o un caso excepcional). La herramienta reina para ver esto es el **Boxplot (Diagrama de Caja)**.



Guía de Interpretación del Boxplot:

1. **La Caja**: Contiene el 50% central de los datos (entre Q1 y Q3).
2. **La Línea horizontal dentro de la Caja**: Es la mediana.
3. **Los Bigotes**: Se extienden hasta 1.5 veces el Rango Intercuartílico ($IQR=Q3-Q1$). **Cualquier dato fuera de esos límites se marca con un punto, es un outlier.**
    a. El "Bigote" superior llega hasta $Q3+1.5\times IQR$.
    b. El "Bigote" inferior llega hasta $Q1-1.5\times IQR$.
4. **Puntos fuera de los bigotes**: Son los Outliers confirmados matemáticamente. Fíjate en **Literatura**. Ese punto rojo en el 2.1 es un outlier. El resto de la clase está entre 7.8 y 9.

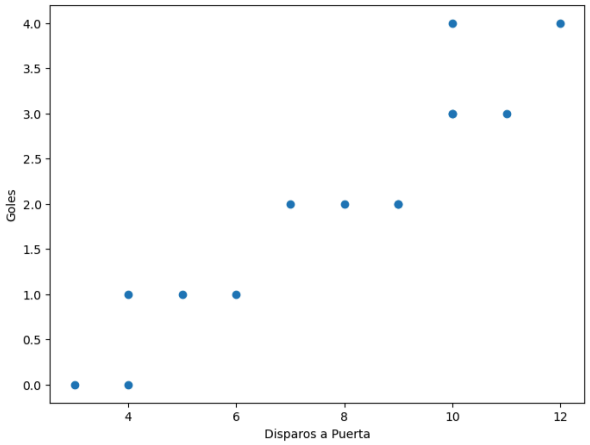
4. Análisis de Distribuciones y Relaciones

La correlación es una medida estadística que cuantifica la fuerza y la dirección de la **relación lineal entre dos variables numéricas**. En términos sencillos: si una sube, ¿qué hace la otra?. Por ejemplo:

- Si un equipo tiene más posesión del balón, ¿suele marcar más goles?
- Si un jugador corre más kilómetros en un partido, ¿tiene más asistencias?

La correlación nos ayuda a encontrar estas relaciones, pero ojo: **¡correlación no significa causalidad!** Que dos variables estén relacionadas no significa que una cause la otra. La causa puede ser una tercera variable "oculta" o ser una simple coincidencia (correlación espuria). *Ejemplo clásico: El aumento de ventas de helados y el aumento de ataques de tiburón están correlacionados, pero la causa es el Calor (verano), no que el helado atraiga tiburones.*

Ejemplo: En la gráfica vemos representadas 2 variables: Eje Y: Goles; Eje X: Disparos a puerta. Podemos ver que cuantos más disparos a puerta, más goles, se visualiza una correlación lineal.



Coeficiente de Correlación (r): Matemáticamente la correlación se puede medir. Existen varios tipos de cálculo, el más utilizado es la Correlación de Pearson. Mide la relación lineal y su valor oscila entre -1 y 1:

- **r = 1:** Correlación positiva perfecta (si una sube, la otra sube en la misma proporción).
- **r = −1:** Correlación negativa perfecta (si una sube, la otra baja proporcionalmente).
- **r = 0:** No existe relación lineal (los puntos están dispersos sin orden).

Pandas permite calcular la correlación de forma masiva entre todas las columnas de un DataFrame mediante el método `.corr(method)`. O bien es posible invocar al método desde una Series (una columna) y pasar como argumento la otra variable.

Existen tres métodos principales:

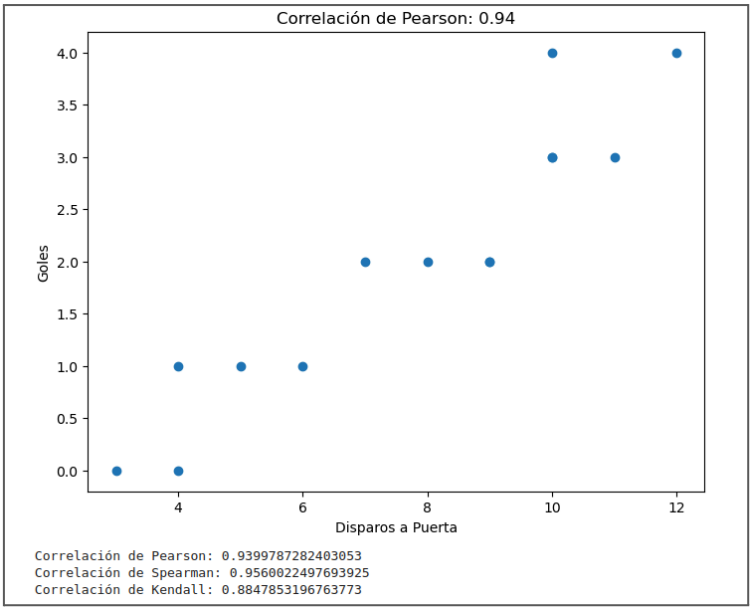
- **Correlación de Pearson** (la más común, por defecto en Pandas): Mide relaciones lineales (como la gráfica anterior).
- **Correlación de Spearman** ('spearman'): Funciona bien cuando la relación no es lineal. Se basa en rangos en lugar de valores reales como en Pearson, lo que lo hace menos sensibles a valores atípicos.
- **Correlación de Kendall** ('kendall'): Parecida a Spearman, pero más robusta en conjuntos de datos pequeños.

Ejemplo: Si analizamos posesión del balón (%) y goles marcados, la correlación de Pearson nos dirá si hay una relación lineal entre ellas. Si analizamos posición en la tabla y número de victorias, la correlación de Spearman puede ser más útil.

```
datos = {
    'Disparos_Puerta': [5, 8, 3, 10, 6, 12, 4, 9, 7, 11, 10, 4, 10, 9],
    'Goles': [1, 2, 0, 3, 1, 4, 1, 2, 2, 3, 4, 0, 3, 2]
}
df = pd.DataFrame(datos)

# Calcular la correlación de Pearson
# Pearson es el método por defecto, no haría falta indicar el método
corr_pearson = df['Disparos_Puerta'].corr(df['Goles'], method="pearson")
# Spearman
corr_spearman = df['Disparos_Puerta'].corr(df['Goles'], method="spearman")
# Kendall
corr_kendall = df['Disparos_Puerta'].corr(df['Goles'], method="kendall")

# Crear un gráfico de dispersión
plt.figure(figsize=(8, 6))
plt.scatter(df['Disparos_Puerta'], df['Goles'])
plt.title(f'Correlación de Pearson: {correlacion:.2f}')
plt.xlabel('Disparos a Puerta')
plt.ylabel('Goles')
plt.show()
```

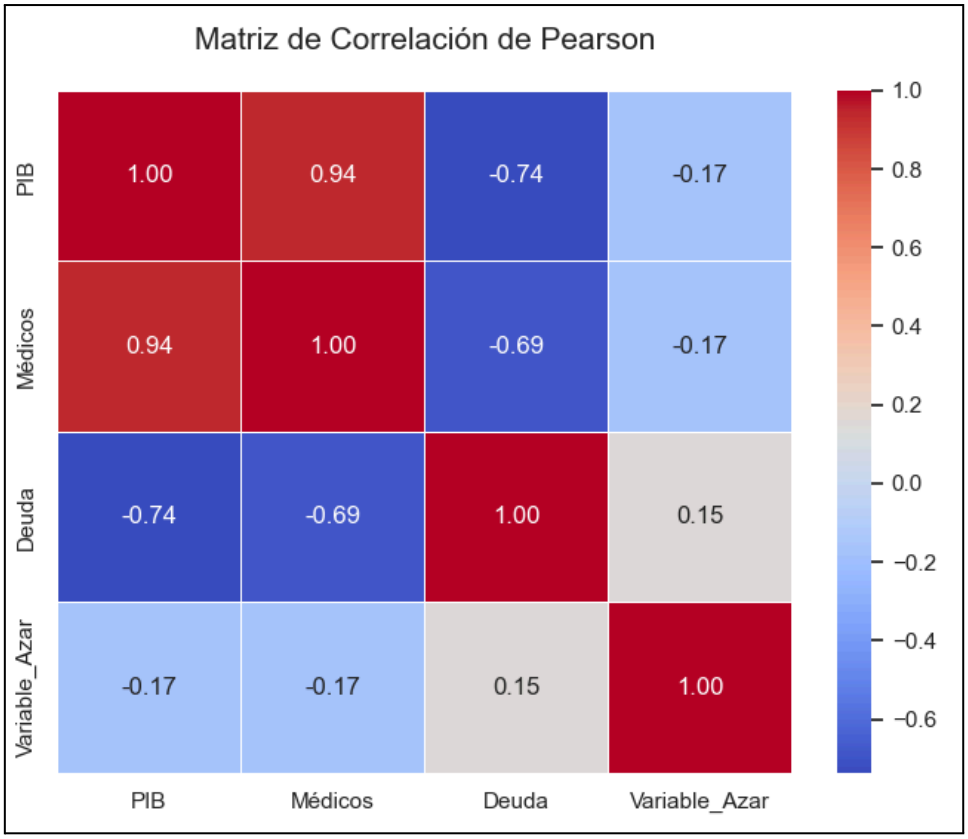


Visualización de la Matriz de correlación: El método `.corr(method)` general una matriz con los valores de correlación de cada par de variables. Para visualizar qué pares de variables están más o menos correlacionadas se muestra la matriz de correlación en un mapa de calor. Cuanto más oscuro el color, más correlación.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Generamos datos artificiales
np.random.seed(42)
n = 100
pib = np.random.normal(50000, 15000, n)
medicos = pib * 0.002 + np.random.normal(0, 10, n)
deuda = -pib * 0.5 + np.random.normal(0, 5000, n)
azar = np.random.normal(50, 20, n)
# 2. Se crea el dataframe a partir de los datos generados
df_corr = pd.DataFrame({'PIB': pib, 'Médicos': medicos,
    'Deuda': deuda, 'Variable_Azar': azar})

# 3. Calculamos la matriz de correlación (Pearson por defecto)
matriz_corr = df_corr.corr(method='pearson')
# 4. Visualización de la matriz de correlación en un Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(matriz_corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('4.1. Matriz de Correlación de Pearson', fontsize=15, pad=20)
plt.show()
```



5. ANOVA

En secciones anteriores hemos visto cómo analizar la relación entre dos variables numéricas mediante la correlación. Sin embargo, a menudo nos enfrentamos a un escenario diferente: queremos saber si una **variable categórica** (por ejemplo comparar diferentes tácticas de fútbol) influye en una **variable numérica** (goles marcados en el partido).

El **ANOVA** es la técnica estadística que nos permite comparar las medias de **tres o más grupos** para determinar si las diferencias observadas entre ellos son estadísticamente significativas o si se deben simplemente al azar (si la táctica influye en los goles).

- **¿Cuándo se usa?**: Se utiliza cuando la variable independiente es categórica (ej: tácticas de juego, tipos de fertilizantes, departamentos de una empresa) y la variable dependiente es numérica (ej: goles, crecimiento de planta, salario).

El concepto fundamental: Variabilidad

ANOVA basa su nombre en que descompone la variabilidad total de los datos en dos fuentes:

1. **Variabilidad entre grupos (Between-groups)**: Qué tan diferentes son las medias de los grupos entre sí.
2. **Variabilidad dentro de los grupos (Within-groups o Residual)**: Qué tan dispersos están los datos dentro de cada grupo individualmente.

Si la diferencia *entre* los grupos es mucho mayor que la dispersión *dentro* de ellos, podemos concluir que el grupo al que pertenece un dato realmente influye en su valor.

Interpretación del P-Valor

El resultado más crítico de un test ANOVA es el **p-valor**:

- **Si $p<0.05$**: Existen diferencias significativas. Al menos uno de los grupos se comporta de forma distinta a los demás (el grupo influye en la variable numérica).
- **Si $p>0.05$**: No hay diferencias significativas. Las pequeñas variaciones en las medias pueden atribuirse al azar y no a la categoría analizada.

> **Nota importante**: ANOVA nos dice *si* hay una diferencia, pero **no nos dice exactamente *qué* grupo es el diferente**. Para eso se requerirían pruebas adicionales llamadas "Post-hoc".

Implementación en Python con `statsmodels`

Para realizar este análisis utilizaremos la librería `statsmodels`, que permite definir modelos estadísticos de forma muy intuitiva mediante fórmulas.

Ejemplo Práctico: Queremos analizar si la formación táctica de un equipo (4-4-2, 4-3-3, 5-3-2) afecta al promedio de goles marcados.

Para calcular el **p-valor** y realizar el análisis de la varianza usamos la librería **statsmodels**.

```python
import statsmodels.api as sm
import statsmodels.formula.api as smf

modelo = smf.ols("goles ~ formacion", data=datos).fit()
anova_resultado = sm.stats.anova_lm(modelo)
```

1. Definimos un modelo de regresión lineal (se avanzará en el tema 4) con **smf.ols. E**n la fórmula de ols, el orden SIEMPRE es el mismo:

   Variable Numérica (Dependiente)~Variable Categórica (Independiente)

   a. "goles ~ formacion" son columnas del DataFrame 'datos' pasado como parámetro:
      i. goles: Variable dependiente numérica (lo que queremos analizar).
      ii. formacion: Variable independiente categórica (el factor que creemos que influye en la cantidad de goles). Antes del nombre de la columna se puede poner C o Q:
         1. C: Si la categoría es un número (1, 2, 3…) C(nombre_col) indica que la variable es en realidad una categoría.
         2. Q: Cuando el nombre de la columna tiene espacios Q("nombre de la columna")
   b. .fit(): Ajusta el modelo a los datos.
2. Calcular ANOVA al modelo ajustado con sm.stats.anova_lm(modelo). El resultado es una matriz de valores.

```python
modelo = smf.ols("goles ~ formacion", data=datos).fit()
anova_resultado = sm.stats.anova_lm(modelo)

print(anova_resultado)
```

```
              df     sum_sq    mean_sq          F    PR(>F)
formacion    2.0  25.266667  12.633333  22.892617  0.000002
Residual    27.0  14.900000   0.551852        NaN       NaN
```

3. Interpretar los valores
   a. **df (Grados de libertad)**
      i. La **formación** tiene 2 grados de libertad (porque hay 3 grupos: 4-4-2, 4-3-3 y 5-3-2 → df = k - 1 = 3 - 1 = 2).
      ii. El **residual** tiene 27 grados de libertad (total de 30 observaciones menos el número de grupos: 30 - 3 = 27).

b. **sum_sq (Suma de cuadrados)**
   i. **25.27** para "formación" indica la variabilidad explicada por la táctica utilizada.
   ii. **14.90** para "residual" es la variabilidad aleatoria o residual.
c. **mean_sq (Media de los cuadrados):** Es la suma de cuadrados dividida por los grados de libertad.
   i. **12.63** para "formación" (25.27 / 2).
   ii. **0.55** para "residual" (14.90 / 27).
d. **F (Estadístico F de Fisher)**
   i. **22.89** es el valor del test F. Valores altos indican diferencias más grandes entre los grupos.
e. **PR(>F) (p-valor): 0.000002** → Es el **p-valor**, que indica la probabilidad de obtener estos datos si no hubiera diferencia entre las formaciones. Como **0.000002 < 0.05**, significa que **hay una diferencia significativa en la cantidad de goles según la formación.**

El ANOVA siempre debe ir acompañado de un Boxplot. Mientras que el ANOVA nos da el rigor matemático, el Boxplot nos permite visualizar esas diferencias de media y la dispersión de los grupos de un vistazo.

Test de Tukey

El Test de Tukey es el complemento indispensable del ANOVA. ANOVA es el "semáforo" que te dice si hay alguna diferencia en el grupo, el Test de Tukey es el "mapa" que te dice exactamente dónde está esa diferencia.

- Si comparas 5 grupos y el p-valor es <0.05, el ANOVA solo te dice: *"Al menos dos de estos grupos son diferentes entre sí"*.
- Pero... ¿cuáles? ¿Es el grupo A distinto al B? ¿O es el E el que es diferente a todos los demás?

El test puede ser ejecutado por la función `pairwise_tukeyhsd` de statsmodels. Es necesario importar (`from statsmodels.stats.multicomp import pairwise_tukeyhsd`).

```python
from statsmodels.stats.multicomp import pairwise_tukeyhsd
# C. Test de Tukey (Comparación múltiple)
tukey = pairwise_tukeyhsd(endog=datos['goles'], groups=datos['formacion'], alpha=0.05)
print("\nTest de Tukey:")
print(tukey.summary())
```

La función requiere tres parámetros fundamentales:

- **endog**: Variable numérica objetivo
- **groups**: Columna o variable que define los grupos o etiquetas de la variable categórica.
- **alpha**: Umbral de error que estamos dispuestos a aceptar. Casi siempre se usa 0.05 (corresponde a un nivel de confianza del 95%).

La función devuelve una tabla con las columnas de la imagen, y tantas filas como parejas de grupos dentro de la variable categórica a comparar:

group1	group2	meandiff	p-adj	lower	upper	**reject**
Grupo A	Grupo B	5.23	0.012	1.20	9.26	**True**
Grupo A	Grupo C	0.45	0.900	-3.50	4.40	**False**

- **group1** y **group2** son los 2 grupos dentro de la variable categórica que se comparan en la fila correspondiente.
- **meandiff**: Es la resta de las medias (group2 - group1). Si es positivo, el `group2` tiene una media más alta que el `group1`.
- **p-adj**: El p-valor corregido. Si es <0.05, la diferencia es real.
- **reject**: Es la columna más importante.
    - **True**: ¡Hay diferencia! Son grupos distintos.
    - **False**: No hay pruebas suficientes. Se consideran "iguales" estadísticamente.

Para nuestro ejemplo, estamos comparando los goles marcados en 3 formaciones distintas. El resultado son las comparaciones entre las tres formaciones.

```
Test de Tukey:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================
group1 group2 meandiff p-adj   lower   upper  reject
-----------------------------------------------------
 4-3-3  4-4-2  -1.6607 0.0044 -2.7994 -0.5221   True
 4-3-3  5-3-2    0.025 0.9986 -1.2292  1.2792  False
 4-4-2  5-3-2   1.6857 0.0099  0.3975   2.974   True
-----------------------------------------------------
```

El análisis: Mirando la columna `reject`, vemos que hay dos combinaciones donde el resultado es **True**. Esto significa que en esos casos la diferencia de goles **no es casualidad**, es estadísticamente significativa.

- **Diferencia real:** Entre la formación `4-3-3` y la `4-4-2`. meandiff -1.66, 4-4-3 marca de media 1.66 goles más que 4-4-2.
- **Diferencia real:** Entre la formación `4-4-2` y la `5-3-2`. meandiff 1.66, 5-3-2 marca de media 1.66 goles más que 4-4-2.
- **Sin diferencia:** Entre la `4-3-3` y la `5-3-2` (**False**). Se comportan de forma casi idéntica en cuanto a goles.
- **Las mejores formaciones** son 4-3-3 y 5-3-2 sin diferencias entre ellas.

6. Análisis de Series Temporales

Una serie temporal es un conjunto de **datos recolectados en intervalos de tiempo**. Cada medición tiene asociada una **marca de tiempo** del instante exacto de la medición u ocurrencia del dato. A diferencia del EDA estándar, aquí el orden es fundamental: cada dato depende, en parte, de los anteriores.

Podemos encontrar series temporales **regulares** que miden los datos a una frecuencia fija (siempre el mismo intervalo de tiempo) o **irregulares** que no tienen un tiempo fijo entre medida y medida.

6.1. Preprocesado con Pandas

1. Pasar en el DataFrame la columna tiempo a tipo datetime.

```
df['Fecha'] = pd.to_datetime(df['Fecha'])
```

2. Establecer como índice (permite filtrado inteligente)

```
df.set_index('Fecha', inplace=True)
```

3. Filtrados por fecha

```
# FILTRADOS CON LOC
# Extrae todo el año
datos_2023 = df.loc['2023']
# Filtrado por mes y año
df.loc["2024-05"]
# Rango de meses
df.loc["2023-04":"2023-09"]
# Slicing de fechas
df.loc["2023-01-01":"2023-06-30"]
```

```
# INDEXACIÓN BOOLEANA y el atributo index
df[(df.index >= "2023-01-01") & (df.index <= "2023-06-30")]
# Filtrado por número de mes
df[df.index.month == 3]   # marzo
# Filtrado por nombre de mes
df[df.index.month_name() == "March"]
# Filtrado por varios meses
df[df.index.month.isin([1, 3, 12])]  # enero, marzo y diciembre
# Filtrado por mes y año concreto
df[(df.index.year == 2024) & (df.index.month == 5)]
# Filtrado por rango de meses
df[df.index.month.between(4, 9)]  # abril a septiembre (todos los años)
```

```
# QUERY
df.query("index.month == 6")
df.query("index.year == 2022 and index.month >= 10")
```

6.2. Transformación y suavizado

Las series suelen ser ruidosas. Si el intervalo de tiempo es muy bajo para el tipo de análisis, podemos tener mucha variabilidad en los datos de la serie temporal. Para suavizar la medida y ver mejor las tendencias podemos aplicar 2 transformaciones:

- **Resampling:** Es una técnica de **agregación**. Lo que haces es "agrupar" todos los datos en un rango (por ejemplo un mes natural) y convertirlos en un único valor (el promedio de ese mes).

    - **Cómo funciona:** Si tienes 30 datos en enero, los sumas y divides por 30. El resultado es **un solo punto** para enero.
    - **Resultado:** El conjunto de datos se **encoge**. Si tenías 365 filas (días), tras un resample mensual tendrás solo 12 filas.
    - **Para qué sirve:** Para ver el rendimiento global de periodos cerrados. Es ideal para informes de ventas o balances mensuales.
    - Ejemplo en pandas (de días a meses), calculando la media del rango:
        - `df.resample('M').mean()`

- **Medias Móviles (Rolling Window):** La **Media Móvil** es una técnica de **suavizado**. No agrupa por meses naturales, sino que crea una "ventana" que se desplaza día a día.
    - **Cómo funciona:** Con una ventana de 30 días, para el día 30 de enero calcula la media de los 30 días anteriores. Para el día 31 de enero, calcula la media de los 30 días anteriores a ese (del 2 al 31).
    - **Resultado:** El número de filas **se mantiene igual**. Si tenías 365 días, sigues teniendo 365 puntos, pero la curva es más suave porque cada punto es el promedio de su "vecindario".
    - **Para qué sirve:** Para eliminar el "ruido" (picos bruscos) y ver la tendencia sin perder la resolución diaria. Es la herramienta favorita de los analistas de bolsa.
    - **Ejemplo de 7 días**: `df[columna].rolling(window=7).mean()`

Por ejemplo, para una serie temporal (es un DataFrame) de cotizaciones del índice IBEX 35, los datos incluidos son:

- Fecha: día de la toma de datos.
- Cierre: valor del IBEX al cierre de la sesión.
- Apertura: valor del IBEX al inicio de la sesión.
- Mayor: Valor más alto que ha tenido en el día el IBEX.
- Menor: Valor más bajo del IBEX.
- Volumen: Millones negociados en la bolsa ese día.

Para importar los datos usamos la librería “yfinance” que accede a la API de Yahoo Finance. Acotamos datos a 2025.

```python
import yfinance as yf
import pandas as pd

# El "ticker" del IBEX 35 en Yahoo Finance es ^IBEX
# Descargamos los datos para todo el año 2025
datos_ibex = yf.download('^IBEX', start='2025-01-01', end='2025-12-31')

# El objeto que devuelve es un DataFrame de pandas
print(type(datos_ibex))
```

```
[*********************100%***********************]  1 of 1 completed
<class 'pandas.core.frame.DataFrame'>
```

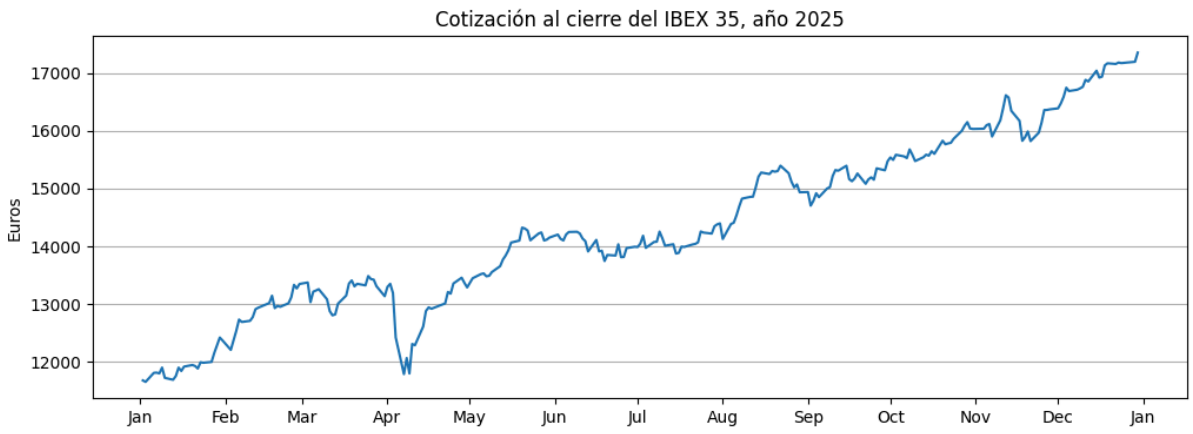
```python
# Vemos el tipo de dato del índice, por asegurarnos
datos_ibex.index
```

```
DatetimeIndex(['2025-01-02', '2025-01-03', '2025-01-06', '2025-01-07',
               '2025-01-08', '2025-01-09', '2025-01-10', '2025-01-13',
               '2025-01-14', '2025-01-15',
               ...
               '2025-12-15', '2025-12-16', '2025-12-17', '2025-12-18',
               '2025-12-19', '2025-12-22', '2025-12-23', '2025-12-24',
               '2025-12-29', '2025-12-30'],
              dtype='datetime64[ns]', name='Date', length=254, freq=None)
```

Hacemos un primer análisis generando un gráfico sobre los valores de cierre:

```python
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

plt.figure(figsize = (12, 4))
plt.plot(datos_ibex['close'])
# Forzamos los 12 meses en eje x
plt.gca().xaxis.set_major_locator(mdates.MonthLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b'))
plt.grid(axis='y')
plt.ylabel("Euros")
plt.title("Cotización al cierre del IBEX 35, año 2025")
plt.show()
```

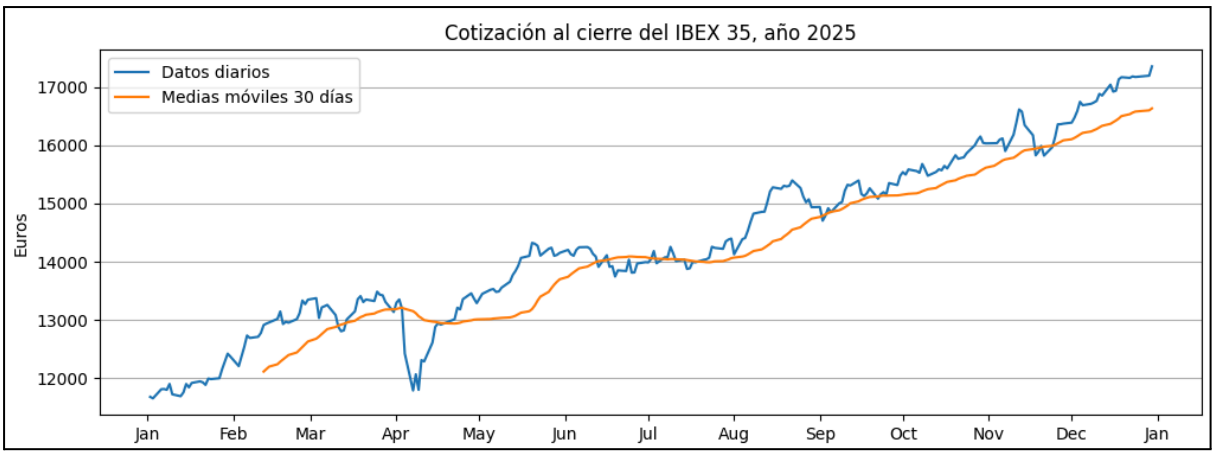


Se puede ver un bajón en la cotización en abril, que se recuperó durante el propio mes antes de iniciar mayo. Dado que es cotización diaria, la línea presenta los propios altibajos de la cotización. Si lo que deseamos es ver la tendencia mensual, tantos altibajos diarios pueden sobrar en el análisis y es mejor procesar los datos.

Para **eliminar las variaciones bruscas** y lograr ver mejor la tendencia vamos a **calcular las medias móviles** de 30 días.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

plt.figure(figsize = (12, 4))
# Línea con datos diarios
plt.plot(datos_ibex['close'], label="Datos diarios")
# Forzamos los 12 meses en eje x
plt.gca().xaxis.set_major_locator(mdates.MonthLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b'))
# Línea con medias móviles
cierre_mmoviles = datos_ibex['close'].rolling(window = 30).mean()
plt.plot(cierre_mmoviles, label="Medias móviles 30 días")
plt.grid(axis='y')
plt.ylabel("Euros")
plt.legend()
plt.title("Cotización al cierre del IBEX 35, año 2025")
plt.show()
```



De esta manera se ve mucho mejor la tendencia anual.

Como puede verse la línea no comienza en enero, ya que se necesitan anteriores (el día actual y 29 anteriores) para calcular la media, y los días del 1 al 29 no tienen 29 días anteriores para ello. Pandas rellena estos valores con nulos (la bolsa cierra los fines de semana).

```
cierre_mmoviles.isna().sum()

np.int64(29)
```

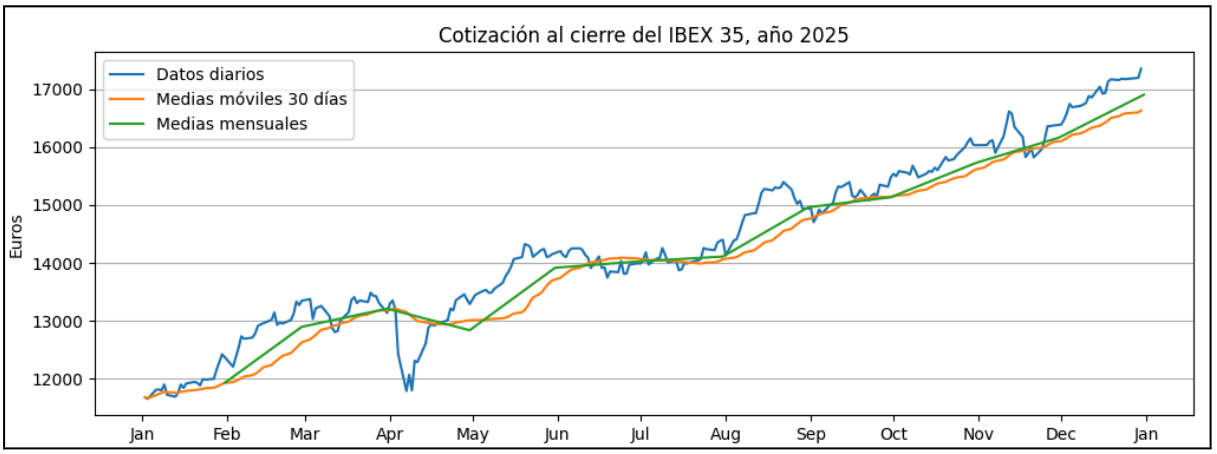
Podemos indicarle que calcule la media de todas formas con los datos que tenga si queremos que comience la línea el día 1 de enero, pero las 29 primeras medias no seran medias móviles de 30 días.

```
# con min_periods le indicamos el periodo mínimo para calcular media
cierre_mmoviles = datos_ibex['close'].rolling(window = 30, min_periods = 1).mean()
```

A continuación comparamos las medias móviles de 30 días con el resampling mensual.

```python
cierre_resample = datos_ibex['close'].resample('ME').mean()
cierre_resample.head()
```

```
Date
2025-01-31    11928.031783
2025-02-28    12895.575000
2025-03-31    13208.295201
2025-04-30    12835.844873
2025-05-31    13915.004743
Freq: ME, Name: close, dtype: float64
```



6.3. Autocorrelación

La autocorrelación es la correlación entre una variable y sus propios valores rezagados en el tiempo, es decir, mide cuánto se parece el valor de hoy al de hace "n" días (retrasos o *lags*). Interpretación de la autocorrelación:

- **Autocorrelación positiva (cercana a 1)**: El valor actual es muy parecido al anterior (tendencia persistente). Por ejemplo, si los precios varían poco en el tiempo.
- **Autocorrelación negativa (cercana a -1)**: Indica que los valores tienden a ser altos cuando los valores pasados fueron bajos, y viceversa. Esto sucede cuando los valores pasados disminuían cuando el valor actual aumenta y viceversa.
- **Autocorrelación cercana a 0**: Sugiere que los valores pasados no tienen una influencia significativa en los valores actuales o futuros. En este caso, los datos se comportan de manera más aleatoria.

Pandas calcula directamente la autocorrelación, indicando en el ejemplo que calcule la autocorrelación con los datos de un día anterior (lag=1), es decir, se calcula la autocorrelación de los datos entre ayer-hoy de todos los días de la serie temporal. En el ejemplo sale una autocorrelación de 1, indicando que los datos de días seguidos son muy similares.

```
# Autocorrelación con el día anterior
correlacion_ayer = datos_ibex['close'].autocorr(lag=1)
print(f"La correlación con el día anterior es: {correlacion_ayer:.2f}")
```

```
La correlación con el día anterior es: 1.00
```

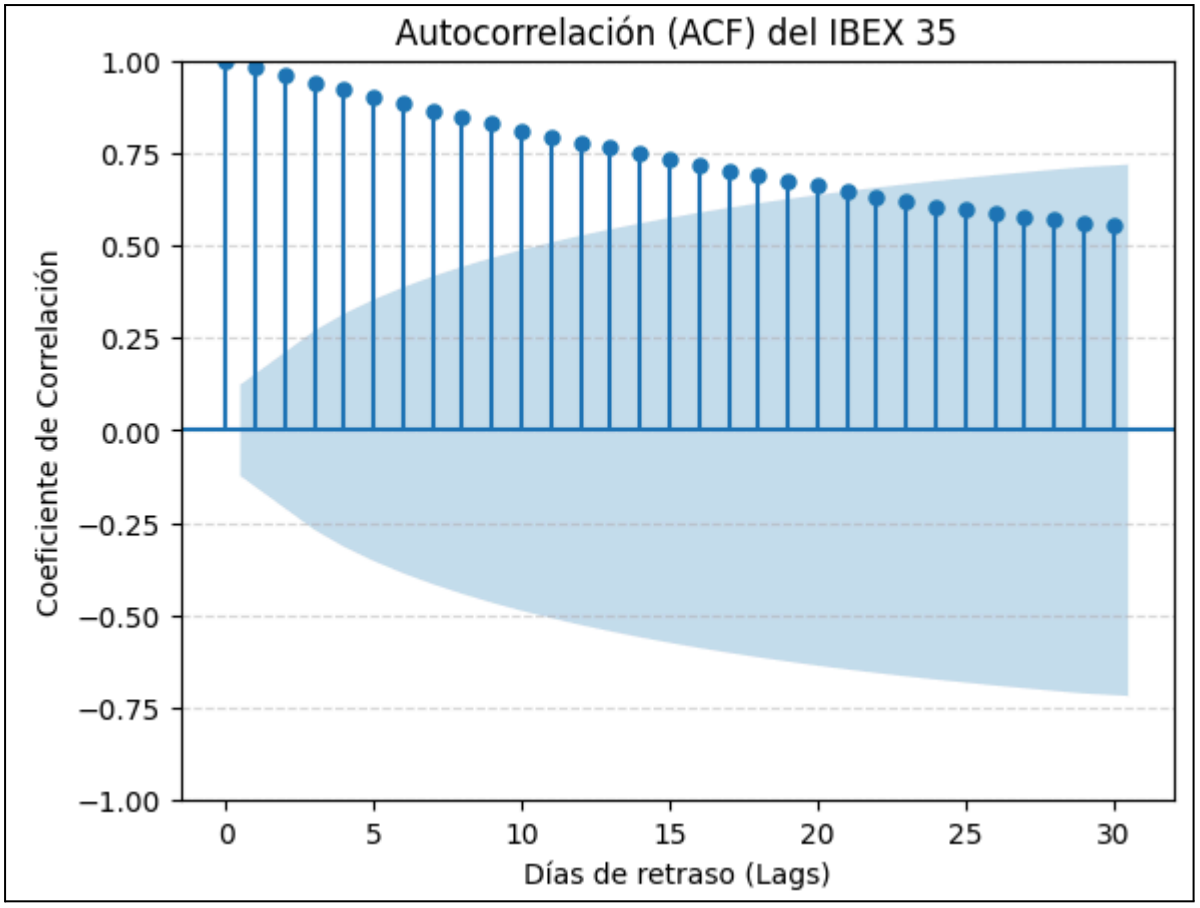
La forma de **visualizar** la autocorrelación es mediante un **correlograma**. El gráfico muestra en el eje X los "retrasos" o *lags* (1 día atrás, 2 días atrás...) y en el eje Y la correlación (de -1 a 1).

- **Barras altas**: Indican que el pasado influye mucho en el presente.
- **Zona sombreada**: Representa el intervalo de confianza. Si la barra sale de la sombra, la correlación es **estadísticamente significativa**.

El ejemplo realizado con statsmodels calcula la autocorrelación de varios lags (de 1 hasta el parámetro lags, en este caso 30).

```
import matplotlib.pyplot as plt
import statsmodels.api as sm


plt.figure(figsize=(10, 5))
# lags=30 significa que queremos ver la influencia de los últimos 30 días
sm.graphics.tsa.plot_acf(datos_ibex['close'], lags=30)
plt.title('Autocorrelación (ACF) del IBEX 35')
plt.xlabel('Días de retraso (Lags)')
plt.ylabel('Coeficiente de Correlación')
plt.grid(True, axis='y', linestyle='--', alpha=0.5)
plt.show()
```


Autocorrelación (ACF) del IBEX 35
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Coeficiente de Correlación
0
5
10
15
20
25
30
Días de retraso (Lags)